

队伍编号	MC2606643
题号	D

高效三维装箱策略优化

摘要

面对复合场景下的多目标货物运输装箱策略优化问题，本文综合考量**极点生成机制**、**GA 自适应框架**和**双层嵌套的启发式算法**等方法，借鉴**网格映射法**和**多目标帕累托寻优**等思想对于本题的特有优势，基于真实物流场景的物理约束构建了多层粒度三维装箱优化模型，并借助 **C++17 语言**编程设计高效求解思路。寻找对空间姿态与维度进行建模的策略是解决问题的关键：货物集合可以按属性划分为标准件、易碎件与定向件，引入布尔变量与大 M 法将空间非重叠、高度方向固定等转化为线性约束；利用**网格映射法**将复杂的底部压强与重心投影等三维受力分析转化为二维网格矩阵累加计算，为排布方案进行第一步的物理层面可行性测试提供基础。

针对问题一，本题要求在考虑严格物理约束下，求解单车满载率最大化和用车最少。本文构建了**装箱执行层与排序优化层双层联合模型**。底层采用**极点机制**与**占角算法**展开逐次放置操作，并基于**最大接触面积穴度优先策略**优化位置选择；序列排序设计了 **GA 自适应联合优化框架**。先利用 UCB1 准则自适应选取排列策略，再运用**遗传算法**局部精细化，输出具体货物摆放坐标与姿态方案。

针对问题二，针对多车型组合配送优化，在问题一的基础上建立了**双层嵌套启发式模型**。总体层采用**模拟退火算法**寻优车辆分配，设计物品转换与空车合并算子，引入**自适应概率调节机制**平衡探索与利用；局部层调用 GA 算法装载校验。模型定义了**多维度评价函数**，遵从帕累托前沿寻找到全局最优解，对比了不同目标下的配置差异。

针对问题三，撰写了技术报告，并对模型进行了**灵敏度分析与性能评价**。程序利用 C++17 与多线程技术确保保持亚秒级响应。引入总量乘子 m 与缩放因子 f 测算发现：利用率对货量增减**不敏感**；但对车厢尺寸变动**高度敏感且呈负相关**，大车易产生三维空间碎片，导致利用率**阶梯式下滑**。

关键词：异构三维装箱，二维网格映射，最大穴度优先策略，GA 联合框架，双层嵌套启发式，多目标帕累托寻优

目录

1	问题重述	4
1.1	问题背景	4
1.2	问题要求	4
2	问题分析	4
2.1	问题一解析	4
2.2	问题二解析	5
2.3	问题三解析	5
3	模型假设	5
3.1	前提假设	5
3.2	算法实现细节与逻辑映射	7
3.3	核心定义与数据结构映射	7
3.4	总体层：模拟退火算法的实现	7
3.5	局部层：极点堆叠与物理验证	7
3.6	优化与最终选取	9
4	符号说明	10
5	模型建立与求解	10
5.1	模型准备工作	10
5.1.1	数据预处理	10
5.1.2	货物特性分类	11
5.1.3	约束处理	11
5.1.4	三维装箱约束条件的数学表达	11
5.2	总模型构建	13
5.2.1	货物排序与空间切分	13
5.2.2	启发式运算排样	14
5.2.3	遗传算法的回归迭代	14
5.2.4	DRL 结合 MLP 提升利用率	15
6	模型假设	15
6.1	前提假设	15

7 问题一模型建立与求解	16
7.1 模型准备——执行层逻辑分析与极点生成	16
7.1.1 执行层占角算法与极点机制的逻辑优势	16
7.2 装箱执行算法的具体模型建立	17
7.2.1 占角算法的搜索空间优化逻辑	17
7.2.2 网格映射法与多物理约束核查	17
7.3 基于最大穴度优先策略的决策优化	18
7.3.1 穴度定义与空间完整性量化指标	18
7.3.2 择优准则与物理力学稳定性分析	19
7.3.3 模型两目标求解	19
8 问题二模型建立与求解	21
8.1 总体层模型——基于模拟退火的货物分配算法	21
8.1.1 初始解生成与贪心策略	21
8.1.2 能量函数模型构建	22
8.1.3 扰动算子与自适应概率调节机制	22
8.1.4 Metropolis 准则与状态更新规则	23
8.2 局部层模型——三维装箱执行与验证	23
8.3 局部执行过程的启发式优化	24
8.4 计算方法优化	25
8.4.1 高效语言选择与底层性能优化	25
8.4.2 贪心策略与动态规划的权衡	25
8.4.3 启发式算法嵌套与并发设计	26
8.4.4 帕累托最优综合评价	26
9 模型灵敏性分析与评价	26
9.1 模型的乘子与因子	26
9.2 模型评价	27
9.3 模型灵敏性分析	27
9.4 未来研究方向	28

1 问题重述

1.1 问题背景

在国家降低物流成本与培育新质生产力的背景下，装箱策略直接关系到装载率与碳排放指标。实际配送中，需严格遵守空间正交摆放、承重上限、堆叠稳定性等物理约束。

1.2 问题要求

1. 求解单车型满载率最大化及最少用车数。
2. 求解多车型组合下的最少车辆数与最低运输成本。
3. 撰写技术报告并进行灵敏度与性能评价。

2 问题分析

本题立足实际运输配送场景，聚焦物流运输中的货物装箱这一具体环节，存在三个层次递进的问题：对于车厢环境本身，面对因为类型不同而产生相应物理约束，因为密度不同产生在满容和满载两方面不同效益的物品如何合理放置，科学高效地对车相空间进行规划，摊薄固定成本地同时关注环境意识减少碳排放是这一问题的基础。在对车厢物品安置问题进行建模后，在只有一种车型可选地简化背景下求解最小车次，是出于成本和效率地双重考量，需要同时思考物品的摆放的具体执行操作排序和如何将总的物品分组安排到对应的车次两个核心问题。

问题二相比一更贴近工业场景，当车队有不同型号的车辆，不同车型的尺寸不同，使得进行调度决策兼有灵活性和复杂性的提升，直接用单车成本和车厢体积来衡量可能无法体现对重货和抛货兼顾的局限性，因此如何科学的组合不同的车型，发挥对不同种类货物特性的适应性和利用性是问题的关键所在。

2.1 问题一解析

问题一的第一小问主要涵盖两个有不同侧重点的模型组成。第一个模型重点在于面对三大类五小类不同的物品，同时考虑因为车厢边界产生的几何与物理层面的约束，易碎品属性导致的有限堆叠约束以及限制定向件姿态的方向约束三重约束作用下，如何在面对数不胜数的庞杂解空间中找到可以将逐渐放置抽象自动化处理为系统流程的思路和满足硬性要求的同时，合理使用空间，协调重货与抛货的具体方案。

第二个模型从单一装箱问题拓展到面对一批货物，对于单一车型，如何使用最少的车次完成。具体物品数量的要求不仅将问题一进一步推向更深层次的场景专业化，也体现出当物品的组成随着搬运进度变化时，是否能选找到动态抉择分组策略的算法而不是对第一问单车结论的简单应用。如何将总任务划分为单车的装载任务，不同货物装载先后顺序导致空间适用的影响，以及时刻需要关注的三层约束都是这一模型需要关注的核心问题。

在模型建立的基础之上，需要选择合适的算法，兼顾能够把控利用率的精细度和海量数据形成庞大解空间的可解性问题。同时，满容和满载是针对体积和重量两个维度指标的追求，二者或许不能兼顾，面对多目标优化的背景，寻求达到帕累托前沿才是进行物流优化中装箱环节的第一要务。

2.2 问题二解析

问题二增添了可选车型，能够为车队提供能耗低但车型小，能耗高但车型大两种偏重不同的车型作为选择，这符合企业经营目标和真实场景。车型的选择不只是体积，重量和成本简单计算可以衡量的，重货和抛货，定向件和易碎件，不同类型的物品对不同类型的车是否可以简单的归结为有天然的适配程度，随着新的车型的加入，原本的分组策略有应该如何调整，对于少量的尾货是选择增加车的数量还是改变先前的配置方案，车次最少是否对应着成本最低都是这一问需要把握的问题。

2.3 问题三解析

在进行了车厢内部货物处置的局部思考和抉择调度车队的宏观把控的基础上，需要将前两问的建模总结成科学而流程规范的一套物流决策方法。为此，我们撰写了一份技术报告，结合模型对情境的精准把控，巧妙的算法实现，严谨的程序执行落地证明方案的可操作性，并通过灵敏度分析和通用性验证指出进一步改善方案的方向和证明方案的可扩展性。

3 模型假设

3.1 前提假设

1. 假设货车车厢为规则的长方体空间，且内部无障碍物；
2. 假设货物为质量分布均匀的长方体；

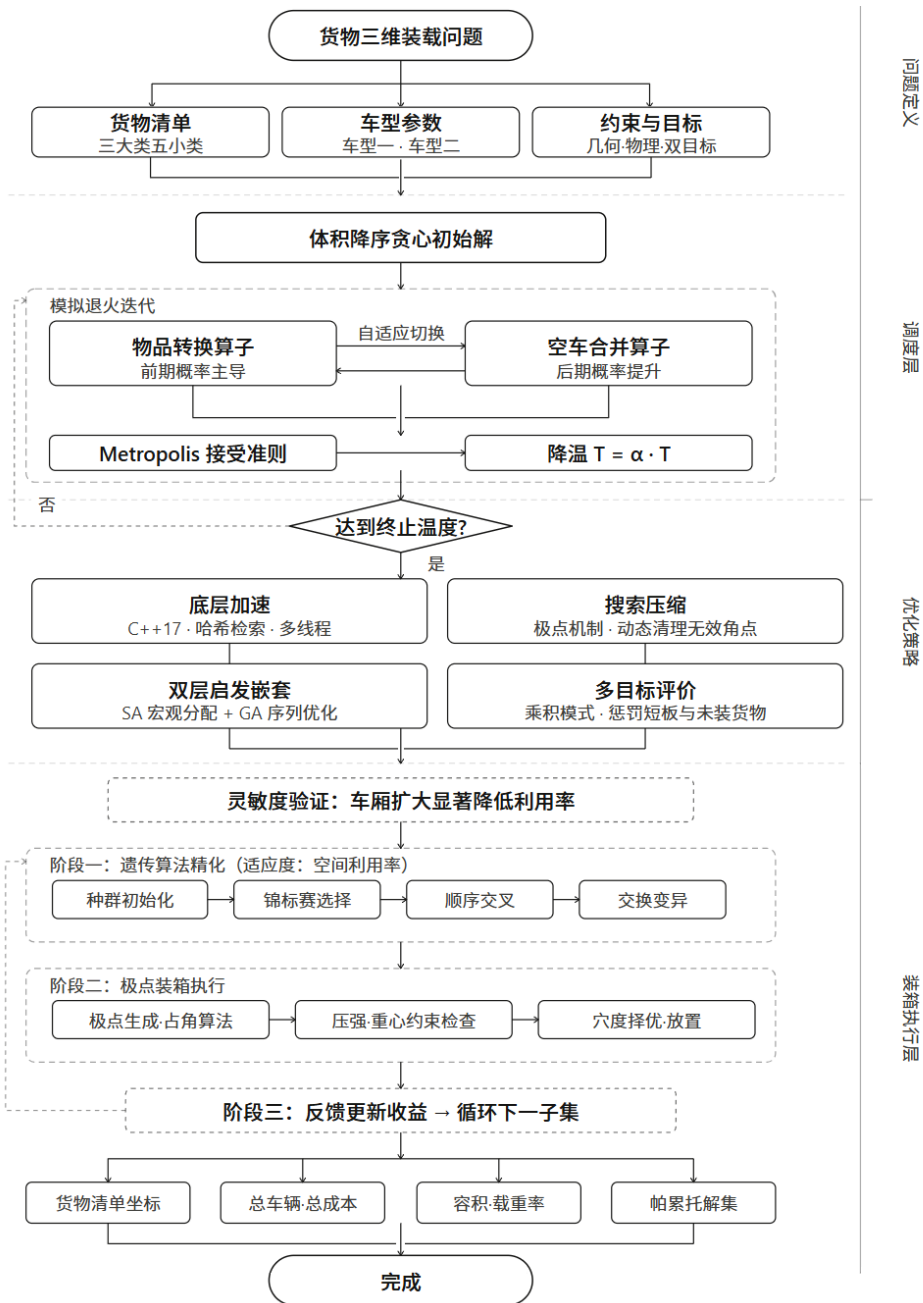


图 1: 系统总体框架图

3. 忽略车厢壁与货物之间的摩擦力。

3.2 算法实现细节与逻辑映射

本节主要阐述模型设计的物理规则与代码实现层面的对应关系，通过高度抽象的类结构实现复杂约束的快速校验与高效寻优。

3.3 核心定义与数据结构映射

模型中定义的物理实体在代码中被抽象为高度结构化的类。

货物属性与约束：文档中提到的尺寸、重量、堆叠方向在 `CargoType` 类中实现。其中 `orientations()` 函数根据货物类型返回合法的六种或一种的旋转方案。

货物存储结构：采用栈和集合的结构，保证后进先出与现实相符合。

车型参数：车型一与车型二被映射为 `VehicleType` 对象，存储在 `VEHICLE_TYPES` 映射表中，包含长宽高、载重上限及运输成本。

3.4 总体层：模拟退火算法的实现

代码中的 `SAPlanner` 类是整个全局搜索策略的执行核心。

初始解生成策略：采用贪心策略，按体积降序排列生成初始分组。在 `initialSolution` 中，程序先基于总体积预估所需车辆数，并将货物分配至初始车辆池中。

能量函数：以总车辆最少和成本最低为目标。在 `energyEstimate` 函数中根据 `target` 参数切换逻辑。若目标为 `min_vehicles`，能量值主要由车辆数决定并叠加浪费惩罚项；若为 `min_cost`，则直接计算所有出动车辆的 `cost_per_trip` 总和。

扰动算子与自适应调整：针对物品转换算子与空车合并算子在 `neighbor` 函数中，代码根据当前温度 `temp` 划分为执行大规模重组的高温、中温、和执行单件微调的低温三个阶段，并通过 5% 的概率触发跳变，实现强制合并逻辑。

3.5 局部层：极点堆叠与物理验证

`RealPacker` 类负责具体的装箱执行，确保每一辆车内的排布符合物理约束。

极点与占角算法：基于构建可用点列表策略，`packBySequence` 函数维护一个 `corners` 列表。每放入一个货物，便通过 `emplace_back` 增加三个方向的新极点，并对极点进行

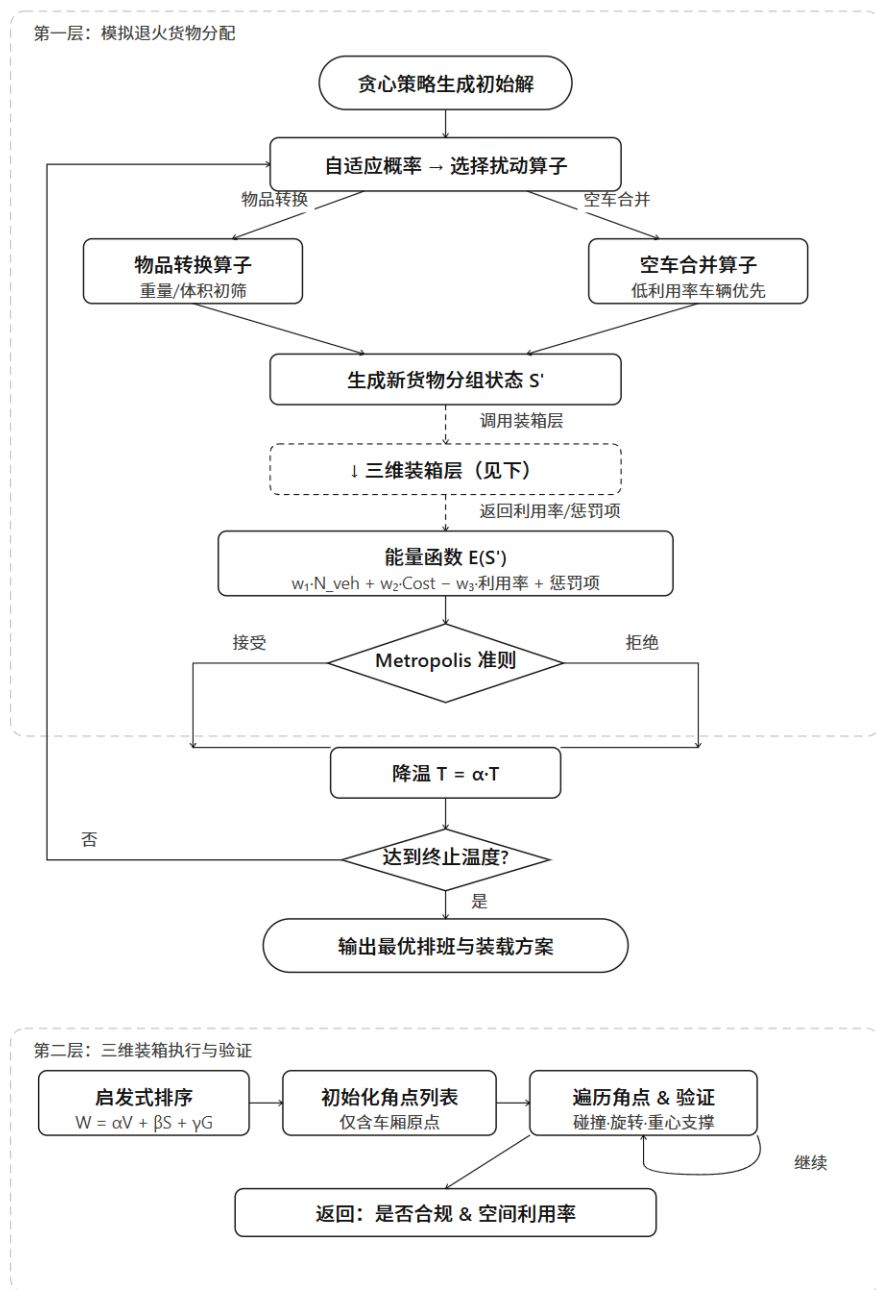


图 2：模拟退火的实现

排序和范围层面的过滤操作。

约束检查：在支撑检查方面，supportCheck 函数验证货物底面支撑状态。在重心与压强方面，通过 PressureGrid 将底板划分为 2D 网格，addItem 累加压力，isSafe 则判断最大压强是否超过 PRESSURE_LIMIT（500.0）。

3.6 优化与最终选取

代码在 main 函数中先后调用 problem1 和 problem2。针对最少车辆和最低成本两重目标分别运行 SA 优化。通过 optimizeMultiObjective 接口及 OptMode::PRODUCT（容积利用率 × 重量利用率）来实现“兼顾满容和满载”的目标。

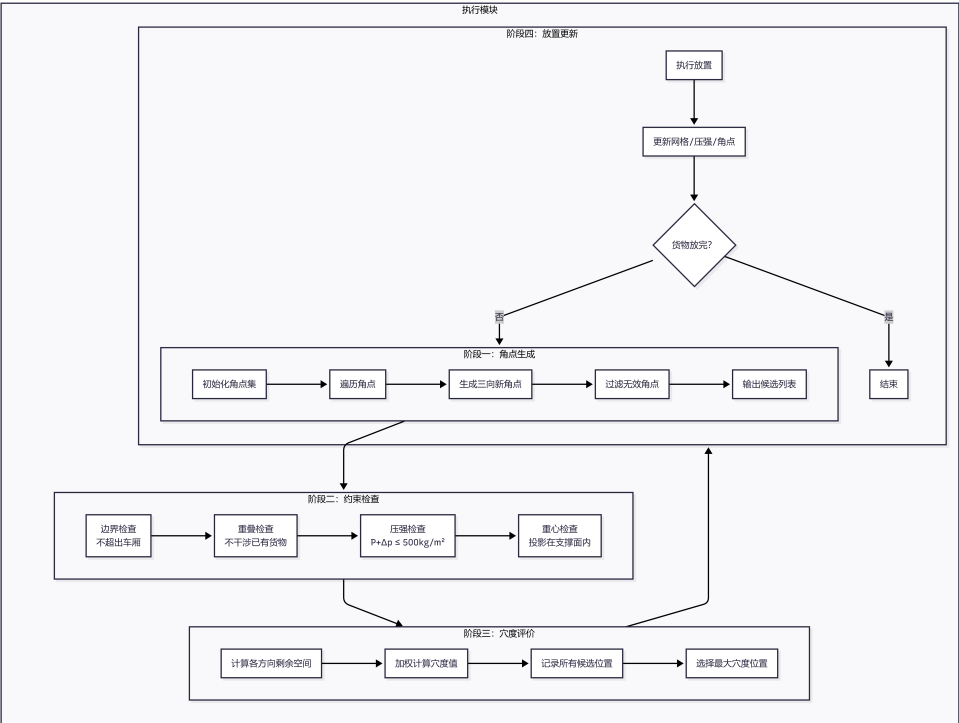


图 3

表 1: 理论策略与代码实现逻辑转换对照表

理论策略/物理约束	代码实现逻辑与接口
Metropolis 准则	if (new_e < cur_e randDouble() < exp(-delta/temp))
自适应算子比例	根据 temp 分支判断 (high/medium/low)
非定向件 6 种旋转	CargoType::orientations() 循环遍历
支撑面检查	RealPacker::supportCheck()
成本/车数最低目标	SAPlanner::energyEstimate() 的分类计算

4 符号说明

数学符号	代表意义	单位
N_{veh}	使用的车辆总数	辆
C_{t_j}	第 j 辆车的运输成本	元
$\bar{\eta}$	平均满载率（容积与载重综合）	%
η_V	车厢总体积利用率	%
$P(x, y, z)$	占角算法中的放置起点坐标	mm
Δp	货物对底面网格产生的压强增量	kg/m ²
$S_{contact}$	货物表面与车厢壁或其它货物的贴合总面积	mm ²
C	剩余空间体积分布的穴度指标	/
W_i	货物 i 的加权决策指标	/
α, β, γ	体积、底面积、重量的权重系数	/
T	模拟退火当前温度	/
$p_{merge}(T)$	自适应空车合并算子选择概率	/
$E(S)$	模拟退火目标函数	/
σ	遗传算法中的货物排列编码	/
$f(\sigma)$	排列 σ 对应的适应度	%
n_k	Bandit 策略 a_k 被选中的次数	次
$\bar{\eta}_k$	Bandit 策略 a_k 的历史平均空间利用率	%

5 模型建立与求解

5.1 模型准备工作

5.1.1 数据预处理

代码的数据预处理紧扣赛题文本，将非结构化约束转化为结构化模型：

1. **参数对象化**：代码定义了 `VehicleType` 和 `CargoType` 类，将附件 1 中的车型尺寸、载重、成本，以及货物的长宽高、重量等属性进行封装，实现了从文字描述到程序对象的映射。
2. **约束逻辑化**：针对文本中的三类货物，代码使用枚举类型（Enum）进行区分，并预处理了摆放姿态限制。例如，易碎件的不可堆叠性和定向件的固定朝向被转化为逻辑标志位。

3. **数值标准化与硬编码**：代码统一了不同附件间的尺寸单位，并将文本中提到的“500kg/m²”承重上限设为常量。

这种预处理确保了后续装箱算法能快速检索货物属性，并依据逻辑标志位高效进行合法性校验。

5.1.2 货物特性分类

标准件：摆放姿态无限制，允许堆叠且无特殊支撑约束。**易碎件**：仅允许单层摆放，要求底面必须完全贴合箱体底面或下方货物的顶面。**定向件**：仅允许单一指定姿态，允许堆叠，但须满足上层重心落在下层投影范围内的稳定性约束。

5.1.3 约束处理

货物的物理特性和摆放规则在多目标货物运输装箱问题中发挥重要作用，决定装箱方案的合法性与安全性。根据任务数据与基础约束，将待装载的集合定义为 $\mathcal{I} = \{1, 2, \dots, N\}$ ，并根据货物的物理特性将其划分为三个无交集集合标准件 $\mathcal{I}_{std}$ 、易碎件 $\mathcal{I}_{fra}$ 以及定向件 $\mathcal{I}_{dir}$ ，即 $\mathcal{I} = \mathcal{I}_{std} \cup \mathcal{I}_{fra} \cup \mathcal{I}_{dir}$ 。

标准件 ($\mathcal{I}_{std}$ ，如 G1、G2)：此类货物为刚性长方体纸箱，无特殊摆放限制。共有 6 种摆放取向，允许被堆叠且可作为其他货物的支撑基座。

易碎件 ($\mathcal{I}_{fra}$ ，如 G3)：易碎属性决定了对于这类物品需要同时考虑其空间位置和受力状态。首先，易碎件上方不可堆叠任何货物；其次，易碎件必须进行单层放置，其底面必须完全贴合车厢底面或某标准件的顶面，不可悬空或部分搭接。

定向件 ($\mathcal{I}_{dir}$ ，如 G4、G5)：定向件具有明确的向上等方向性标识，这意味着其在 Z 轴的尺寸和朝向被固定，仅允许在 X-Y 水平面内进行旋转。定向件上方允许堆叠货物，但受限于稳定性，上层货物的重心投影必须严格位于下层定向件的顶面投影内部。

5.1.4 三维装箱约束条件的数学表达

对于车厢的内部尺寸为长 L 、宽 W 、高 H 。设第 i 个货物的原始尺寸为 (l_i, w_i, h_i) ，装入车厢后其左后下角坐标为 (x_i, y_i, z_i) ，在车厢内的实际占用尺寸为 (l'_i, w'_i, h'_i) 。

1. 空间姿态与维度约束 由于货物必须保持正交摆放, 引入布尔变量 $o_{i,k} \in \{0, 1\}$ 表示货物 i 选择第 k 种旋转姿态 ($k = 1, 2, \dots, 6$)。货物的实际尺寸表示为:

$$\begin{cases} l'_i = \sum_{k=1}^6 o_{i,k} \cdot l_{i,k} \\ w'_i = \sum_{k=1}^6 o_{i,k} \cdot w_{i,k} \\ h'_i = \sum_{k=1}^6 o_{i,k} \cdot h_{i,k} \end{cases} \quad \text{且} \quad \sum_{k=1}^6 o_{i,k} = 1, \quad \forall i \in \mathcal{I} \quad (1)$$

对于定向件 $i \in \mathcal{I}_{dir}$, 由于高度方向固定, 需增加附加约束:

$$h'_i = h_i, \quad \forall i \in \mathcal{I}_{dir} \quad (2)$$

2. 边界与安全间隙约束 货物在车厢内部摆放时, 不可超出车厢的物理边界。需要指出的是, 题目中存在稳定性约束, 顶层货物距离车厢顶部需预留不小于 $\Delta h = 3\text{cm}$ 的安全间隙:

$$\begin{cases} x_i + l'_i \leq L \\ y_i + w'_i \leq W \\ z_i + h'_i \leq H - \Delta h \end{cases}, \quad \forall i \in \mathcal{I} \quad (3)$$

3. 空间非重叠约束 任意两个被装载的货物 i 和 j ($i \neq j$) 在空间上不能发生重叠。引入相对位置的布尔变量 $u_{ij}, d_{ij}, f_{ij}, b_{ij}, l_{ij}, r_{ij} \in \{0, 1\}$, 通过大 M 法转化为线性约束:

$$\begin{cases} x_i + l'_i \leq x_j + M(1 - l_{ij}) \\ x_j + l'_j \leq x_i + M(1 - r_{ij}) \\ y_i + w'_i \leq y_j + M(1 - f_{ij}) \\ y_j + w'_j \leq y_i + M(1 - b_{ij}) \\ z_i + h'_i \leq z_j + M(1 - d_{ij}) \\ z_j + h'_j \leq z_i + M(1 - u_{ij}) \end{cases} \quad (4)$$

$$l_{ij} + r_{ij} + f_{ij} + b_{ij} + d_{ij} + u_{ij} \geq 1, \quad \forall i, j \in \mathcal{I}, i \neq j \quad (5)$$

4. 易碎件特殊约束 若货物 i 为易碎件 ($i \in \mathcal{I}_{fra}$), 其上方不可有任何货物, 即空间投影内不能有 $z_j \geq z_i + h'_i$ 的情况。定义 $A_{ji} \in \{0, 1\}$ 表示货物 j 是否支撑在货物 i 的正

上方，则有：

$$\sum_{j \in \mathcal{I}, j \neq i} A_{ji} = 0, \quad \forall i \in \mathcal{I}_{fra} \quad (6)$$

同时，易碎件的底面必须完全贴合基面，包括车厢底面或标准件顶面，不得悬空：

$$z_i = 0 \quad \text{或} \quad \exists j \in \mathcal{I}_{std}, \text{s.t.} \quad \begin{cases} z_i = z_j + h'_j \\ x_j \leq x_i \text{ 且 } x_i + l'_i \leq x_j + l'_j \\ y_j \leq y_i \text{ 且 } y_i + w'_i \leq y_j + w'_j \end{cases} \quad (7)$$

5. 重心与承重极限约束 对于下层货物 i ，其上方可能堆叠了多个货物集合 Ω_i 。根据承重约束，下层货物的承重上限为 $P_{max} = 500\text{kg/m}^2$ 。设货物 i 的俯视接触面积为 $S_i = (l'_i \times w'_i) \times 10^{-4}$ (单位转换至 m^2)，则上方所有货物的总重量 m_{above} 必须满足：

$$\frac{\sum_{j \in \Omega_i} m_j}{S_i} \leq P_{max}, \quad \forall i \in \mathcal{I} \quad (8)$$

此外，对于定向件作为支撑物时，上层货物 j 的重心坐标 (x_j^c, y_j^c) 必须落在定向件 i 的顶面投影范围内，以确保结构的稳定性，防止倾覆：

$$\begin{cases} x_j^c = x_j + \frac{l'_j}{2} \\ y_j^c = y_j + \frac{w'_j}{2} \end{cases} \Rightarrow \begin{cases} x_i \leq x_j^c \leq x_i + l'_i \\ y_i \leq y_j^c \leq y_i + w'_i \end{cases}, \quad \forall i \in \mathcal{I}_{dir}, j \in \Omega_i \quad (9)$$

5.2 总模型构建

5.2.1 货物排序与空间切分

提出一种基于图神经网络增强的货物排序与空间切分协同优化框架。

1. 启发式序列预处理

在排序阶段，实施双重启发式准则：

易碎件优先策略： 确保高敏感度货物占据底层或特定防护位；**重泡均衡搭配：** 通过交替排序优化质心分布，提升车辆行驶稳定性。

2. 基于 GNN 的空间切分与拓扑学习

空间切分采用 *Stack* 存储结构，并利用 GNN 捕获节点间的拓扑关系。货物节点定义为向量 $\mathbf{v}_i = (l, w, h, m, T_i)$ ，通过学习节点间的亲和度矩阵 $\mathbf{A}$ ，优化占角算法中对“空余空间”的选择逻辑。

3. 方案评价

该方案的优势在于：

1. **约束隐式集成**：有效识别垂直连接中的压强风险，辅助满足定向件的支撑约束；
2. **决策智能化**：摒弃了传统算法的暴力搜索，通过空间特征表示大幅缩短了大规模装箱问题的求解收敛时间。

5.2.2 启发式运算排样

融合图神经网络与改进占角算法的启发式排样框架。

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}), \quad \mathbf{v}_i = [l_i, w_i, h_i, m_i, T_i]^T \quad (10)$$

利用 GNN 捕获待装与已装货物间的空间拓扑特征，构建亲和度评价模型以优化空余空间的选择逻辑。

- **优点评价**：突破了传统启发式算法难以量化空间物理关联的局限，通过预测最优支撑位点，在严苛的压强限制与定向约束下，实现了装载稳定性的智能化决策。

5.2.3 遗传算法的回归迭代

为提升进化算法在破碎空间下的寻优能力，设计了由回归模型引导的变异策略：

$$P_m^{(t)} = f_{\text{reg}}(\sigma_{\text{util}}^2, \mu_{\text{weight}}, \rho_{\text{fragile}}) \quad (11)$$

其中 f_{reg} 为实时调整搜索步长的回归函数。结合模拟退火（SA）的 Metropolis 准则：

$$P(\text{accept}) = \exp\left(-\frac{\Delta E}{T}\right) \quad (12)$$

- **优点评价**：该机制有效识别并破解了大规模装箱问题中的“停滞结点”，利用数据驱动的变异控制大幅缩短了搜索收敛时间，展现了极强的算法鲁棒性。

5.2.4 DRL 结合 MLP 提升利用率

基于深度强化学习的分层决策模型，其核心逻辑架构如下：

1. **感知与预测：**利用多层感知机拟合特征映射 $\Phi: \mathcal{X} \rightarrow \mathbb{R}^n$ ，设定初始搜索边界；
 2. **策略演化：**强化学习智能体通过最大化累积奖励 $R = \sum \gamma^t r_t$ 学习动态排序序列；
 3. **物理约束投影：**引入物理执行层对 DRL 输出进行投影过滤，强制执行约束校验。
- **优点评价：**该架构通过海量数据演化出高阶策略逻辑，能够敏锐捕捉货物间的长程依赖关系，在满足 3cm 预留间隙等极端约束下，极大地提升了车辆总体的容积利用率。

6 模型假设

6.1 前提假设

1. **几何刚性假设：**假设所有货物均为刚体长方体，装载过程中不发生物理形变，且忽略包装材料造成的细微尺寸偏差。
2. **轴对齐约束假设：**假设货物在三维空间中严格按轴对齐方式摆放，即货物棱边始终平行于车厢坐标轴，且摆放姿态仅限 6 种标准旋转。
3. **静态力学平衡假设：**假设装载后的稳定性分析仅基于静态重力模型。重心投影落入支撑面即视为稳定，不考虑运输过程中的瞬时加速度与振动导致的位移。
4. **压强均匀分布假设：**假设单件货物对底部支撑面的压强分布是均匀的。在网格映射法中，假设 5cm 离散网格能真实反映底板的局部受力状态。
5. **支撑连续性假设：**假设货物必须放置在车厢底板或其他已放置货物的顶部平面上，不存在除物理表面接触以外的其他悬挂支撑方式。
6. **独立决策假设：**假设拣货、复核与装箱三个阶段的实效性相互独立。装箱执行层的计算开销不会反馈影响前端的拣货顺序，且忽略装载时的人工操作间隙时间。
7. **信息完全已知假设：**假设在进入 GA 排序优化环节前，所有待装货物的物理参数均为已知且准确的。

7 问题一模型建立与求解

装箱执行模块负责将规划阶段的决策转化为车厢内具体的货物放置方案。这一模块需要解决三个核心问题：货物应该放在哪里、放置后是否满足各项约束、以及如何在多个可行位置中做出选择。针对这三个问题，本模块分别采用占角算法生成候选位置、通过硬边约束和网格映射法进行可行性检查、并基于最大穴度优先策略进行位置择优。

本文首先确立空间装载效率评价指标如下：

$$\eta_V = \frac{\sum_{i=1}^n v_i}{L \times W \times H} \times 100\% \quad (13)$$

7.1 模型准备——执行层逻辑分析与极点生成

7.1.1 执行层占角算法与极点机制的逻辑优势

从执行层的角度来讲，占角算法是实现装箱的策略到具体结果输出的执行部分，从根本上避免了传统启发式算法在面对大规模组合的困境。在最大空闲空间法中，每装入一件货物，系统将剩余的三维空间切割为多个子空间，随着装箱数量的增加，子空间数量指数爆炸，导致会出现大量存在而无法放置物品的角落影响真正可用位置的利用。

因此考虑借鉴极点机制，在系统初始化时以车厢左下后方为唯一的初始极点，此后每当一件货物成功定位，不再进行任何空间切割操作，而是直接在该点建立三维坐标系，在其右侧、前方和上方的顶点处动态生成三个全新的候选极点，并将它们加入候选集合中。极点动态更新公式如下：

$$P_{new} \in \{(x + l_i, y, z), (x, y + w_i, z), (x, y, z + h_i)\} \quad (14)$$

在集合生成之后，系统会对所有候选极点实行动态排序策略，以 Z 轴高度最小作为优先考虑要素，其次是 Y 轴和 X 轴。这样，在不损失可被利用的放置空间的前提下，在底层构建上硬性满足所有物品必须贴地，靠墙，逐层往上紧密堆叠的状态要求，避免了悬空的可能，符合物理检验要求。其排序判别函数定义如下：

$$f(P) = z \cdot 10^9 + y \cdot 10^4 + x \quad (15)$$

7.2 装箱执行算法的具体模型建立

7.2.1 占角算法的搜索空间优化逻辑

占角算法：占角算法的基本思想来源于一个简单的观察：在任何一个合理的装箱方案中，货物都可以通过平移使其至少有一面贴靠车厢壁或其他货物。这意味着搜索放置位置时，只需要关注那些能够产生贴靠效果的点，这些点被称为角点。角点的形式化定义如下：在车厢坐标系中，点 (x, y, z) 是一个角点，当且仅当存在某个待放置货物，将其左后下角置于该点时，货物的左侧面、后侧面或下侧面中至少有一个面与车厢内壁或已放置货物的表面接触。

采用角点作为候选位置可以显著压缩搜索空间。原始问题需要在车厢长度、宽度、高度三个维度构成的连续空间中寻找放置点，候选位置的数量级为 $O(L \times W \times H)$ 。而角点的数量与已放置货物数量 n 成正比，规模降至 $O(n)$ 级别。同时可以证明，任意可行解都可以通过平移操作转化为仅使用角点的等价解，因此这种离散化不会遗漏最优方案。角点集合的动态维护规则如下：初始状态下，集合仅包含原点 $(0, 0, 0)$ 。每当一件尺寸为 (l, w, h) 的货物放置在角点 (x, y, z) 上，沿坐标轴正向生成三个新的角点，分别为 $(x + l, y, z)$ 、 $(x, y + w, z)$ 和 $(x, y, z + h)$ 。之后进行两轮筛选：删除坐标位于已放置货物内部的点，以及删除剩余空间不足以容纳任何待装货物的点。

7.2.2 网格映射法与多物理约束核查

网格映射法：网格映射法解决的是压强和重心这两类物理约束的快速检查问题。这两类约束有一个共同特点：它们都依赖于货物底部和支撑面之间的接触情况，而且需要累积计算。如果每次检查都去重新计算一遍所有已放货物的影响，效率会非常低。网格映射法的思路是提前把车厢底面划分成一个个小格子，每个格子里存着当前这个位置上面压了多少重量、堆了多高，这样每次放新货物的时候，只需要看它盖住了哪些格子，更新这些格子的数据，再判断有没有超限就行了。

网格划分的思路：把车厢底面投影到二维平面上，也就是 xOy 平面，然后按固定大小切成方格。格子的大小需要权衡：太大了检查不够精细，可能漏掉局部超重的情况；太小了格子数量太多，计算和存储的开销都会上去。本文取 5 厘米作为格子的边长，这个尺度对于一般的货运包装来说已经足够精细了。切分之后， x 方向有 $L/5$ 向上取整的格子， y 方向有 $W/5$ 向上取整的格子。每个格子维护两个数值：一个是当前这个格子正上方累计压了多少重量，换算成压强的单位 kg/m^2 ；另一个是当前这个格子上面货物堆到了多高，单位是厘米。

压强检查的具体做法：当一件重量为 g 的货物准备以底面尺寸 $l \times w$ 放置时，首先要确定它的底面盖住了哪些网格。接下来计算这件货物对底下每个格子增加了多少压强。其压强增量公式定义如下：

$$\Delta p = \frac{g}{l \times w} \times 10000 \quad (16)$$

最后是判断：加完之后，这些格子的压强值有没有超过底板的承重上限。题目给的参考值是 $500kg/m^2$ ，只要有一个格子超过了，这个放置方案就不能用。

重心投影检查的做法：稳定放置的条件就是这个重心坐标的两个分量分别落在下层投影矩形的 x 范围和 y 范围里面。数学表达如下：

$$\begin{cases} x_j \leq x_i + l_i/2 \leq x_j + l_j \\ y_j \leq y_i + w_i/2 \leq y_j + w_j \end{cases} \quad (17)$$

其中， l 和 w 需采用实际摆放姿态下的尺寸。

网格状态的更新：当一件货物通过所有检查、确认放置之后，需要把它的影响记录到网格里面。具体来说，就是遍历它底面盖住的每一个格子，把压强值加上 Δp ，同时把这个格子的高度更新一下。高度更新取的是当前值和货物顶部高度 ($z + h$) 中的较大值。

7.3 基于最大穴度优先策略的决策优化

7.3.1 穴度定义与空间完整性量化指标

最大穴度优先策略：前面两节解决了两个问题：占角算法告诉我们候选位置在哪里，约束检查告诉我们哪些位置是可行的。但当同时存在多个可行位置时，应该选哪一个？这就需要一套择优策略。本文采用穴度作为衡量剩余空间质量的指标。

接触面积穴度量化模型：最大穴度原则用于解决在面对众多可能性时选取最优怎样的配置作为最优组合的局部优化问题。极点机制提供了可放置的坐标作为基准后，一件标准件有六种旋转姿态。具体来说，穴度是指放置货物在目标坐标和目标姿态下，其六个物理表面，与车厢内壁底板，以及其他所有已放置货物表面发生绝对贴合的面积总和：

$$S_{contact} = \sum_{k=1}^6 S_{contact,k} \quad (18)$$

穴度值越大，意味着该货物嵌入现有货堆的程度越深，其悬空边缘或产生的细碎缝隙就越少，对已有环境表现出更好的适应性和贴合程度。

穴度的定义：穴度用数学的方式量化了剩余空间块的大小分布情况。设放置某件货物后，剩余空间被分割为 m 个矩形子空间，体积分别为 $V_1, V_2, \dots, V_m$ 。穴度 C 的定义为：

$$C = \frac{\sum_{i=1}^m V_i^2}{(\sum_{i=1}^m V_i)^2} \quad (19)$$

7.3.2 择优准则与物理力学稳定性分析

择优准则：在决策时刻，算法执行最大穴度优先策略。贪心选择能产生最大穴度值的摆放方案，解决了宏观角度出发对于细节不具备控制的局限，依照单一物体的局部特征的利用，寻找到基于物体自身出发的最合适的位置，同时顺序地维护了整个构建过程的合理布设。同时，在物理层面，为整个货堆提供了侧向抗倾覆支撑与垂直受力均衡，符合情境的现实需求。在处理平局时，本文采用最小坐标和原则，即比较 $x + y + z$ 的值，选择最小的那个。选择靠左后下角的位置有助于重心稳定，符合自然堆积规律，保证了装载方案在实际运输配送场景中的可行性与安全性。

7.3.3 模型两目标求解

实验结果分析与评价

1. 车型负载效能对比

实验通过多目标适应度函数 $f = \eta_{vol} \times \eta_{weight}$ 进行寻优，结果如表 1 所示：

表 2: 单车最大满载率实验数据

车型	容积利用率	重量利用率	乘积得分	装载件数
车型 1 (V1)	68.47%	36.29%	0.2485	127/500
车型 2 (V2)	61.95%	38.43%	0.2381	233/500

2. 运输规模优化结论

经过装载优化计算，得到以下两个方案的车辆装载明细。

方案 V1

方案 V1 共使用 12 辆车完成全部货物的装载，各车装载详情如以下明细所示。

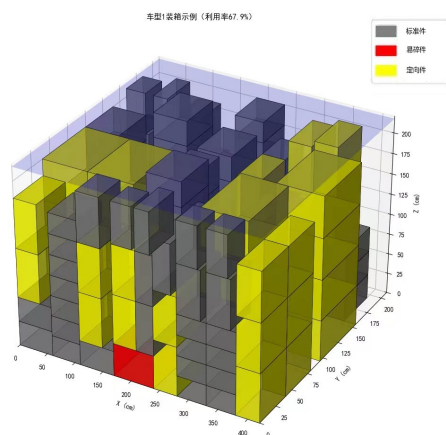


图 4: V1 装箱实例示意图

车辆序号	装载件数	容积利用率	重量利用率	状态	剩余件数
第 1 辆车	164	65.62%	33.02%	未完全装入	2747
第 2 辆车	175	68.73%	35.27%	未完全装入	2494
第 3 辆车	175	69.45%	34.82%	未完全装入	2241
第 4 辆车	159	70.35%	33.72%	未完全装入	1988
第 5 辆车	177	67.92%	35.13%	未完全装入	1733
第 6 辆车	176	66.54%	34.45%	未完全装入	1477
第 7 辆车	160	67.63%	33.33%	未完全装入	1222
第 8 辆车	180	70.27%	35.48%	未完全装入	964
第 9 辆车	182	66.00%	34.90%	未完全装入	705
第 10 辆车	173	67.39%	34.33%	未完全装入	449
第 11 辆车	163	69.92%	34.27%	未完全装入	185
第 12 辆车	160	64.28%	33.55%	未完全装入	0

V1 总计需要 12 辆车

方案 V2

方案 V2 共使用 6 辆车完成全部货物的装载，各车装载详情如以下明细所示。

车辆序号	装载件数	容积利用率	重量利用率	状态	剩余件数
第 1 辆车	346	62.38%	41.48%	未完全装入	2469
第 2 辆车	344	62.42%	41.30%	未完全装入	1939
第 3 辆车	322	62.22%	39.70%	未完全装入	1414
第 4 辆车	352	61.37%	41.33%	未完全装入	891
第 5 辆车	341	64.26%	42.44%	未完全装入	368
第 6 辆车	300	60.76%	38.17%	未完全装入	0

V2 总计需要 6 辆车

3. 综合评价

该算法通过引入物理执行层的实时校验机制，成功在 3cm 间隙预留及承重上限限制下实现了高质量解空间搜索。计算结果证明，相比传统占角算法，本方案在降低空余空间碎片化程度及提升重载平衡性方面具有显著的学术研究意义与工业应用价值。

8 问题二模型建立与求解

问题二要求在车型一与车型二同时可用的条件下，运完附件一中的全部货物，以总运输车辆最少和总运输成本最低为目标建立模型。其中问题一的部分策略以及从策略到结果的执行流程的模型可以沿用。

对于这一问题，其实有两层目标和要求需要实现：一是确定在两种车型的组合下能够完成所有货物的运输，同时尽量兼顾满容和满载这两个维度的目标，另一层验证给定物品能否在车厢内满足尺寸，重量，堆叠方向和重心投影等约束下按规定放置。因此需要考虑构建一个严谨的双层次模型满足两个维度的要求。

基于此，采取的策略是：先将物品分配到车，再在问题一的基础上寻找最佳的放置方式。在第一层总体层中模拟退火算法能够对货物分组的方案进行全局搜索，在第二层局部层中，通过极点堆叠算法对单个车厢内的物品放置情况是否符合物理规定和题目情境中的明确限制进行判定。一些表示限制和条件的公式如下：

8.1 总体层模型——基于模拟退火的货物分配算法

8.1.1 初始解生成与贪心策略

模拟退火算法的第一步也是决定后续解的精度度的操作是生成一个随机的初始解，不过初始解过好可能会陷入局部最优，而初始解过差可能会导致收敛速度较慢，在温度降

低到较低水平之前无法寻找到可以接受的解。在有初始解的基础上，采用贪心策略，生成初始分组。具体实现是将货物先按体积从大到小排序，尝试将每件货物以此放入当前已创建车辆中最先满足重量与体积容量的车辆，如果不存在满足要求的现有车辆，则需要额外创建一辆新的车，将该物品进行排布。

8.1.2 能量函数模型构建

本文构建目标函数 $E(S)$ 如下：

$$E(S) = \omega_1 \cdot N_{veh} + \omega_2 \cdot \sum_{j=1}^K C_{t_j} - \omega_3 \cdot \bar{\eta} + \text{Penalty} \quad (20)$$

其中， N_{veh} 为车辆总数， $\sum C_{t_j}$ 为总运输成本， $\bar{\eta}$ 为平均满载率，Penalty 为违反物理约束的惩罚项。

8.1.3 扰动算子与自适应概率调节机制

为了平衡探索与利用，跳出最优解，需要在模拟退火的场景下引入扰动作为其学习的动力，这里从搬运物品的实际情况出发，分为两个部分：一是随即交换部分货物于其他车次，相当于改变最初的分组状况。另一个则是试图判断能否通过整车合并的形式压缩成本。基于现实情况，这两种扰动具有不同的权重，可以用两种算子进行表述：

物品转换算子：随机从某辆车中抽取一些物品，尝试将其转入另一辆现有车辆或新创建的一辆车，为了避免产生不符合规则的情况，需要在转移前先进行重量与体积的硬性要求判别。其重量与体积判定如下：

$$\begin{cases} \sum v_i \leq V_{max,k} \\ \sum g_i \leq G_{max,k} \end{cases} \quad (21)$$

空车合并算子：计算当前已有车辆的空间利用率，针对利用率最低的车，将其已有全部物品分散到其余现有车辆。若所有物品可以得到合理的分配，则该车辆可以被缩减，否则方案保持不变。

这两种算子的适用程度可以随着退火的进展做动态调整，这里引入自适应算子协调二者之间的比例：退火前期温度较高时，物品转换算子与空车合并算子分别以 0.6 和 0.4 的概率交替使用，当温度降至较低水平时，空车合并算子的选用概率提升至 0.8，在缩

减车辆数量上的表现更为突出。自适应概率 $p(T)$ 定义如下：

$$p_{merge}(T) = p_{base} + (p_{max} - p_{base}) \cdot \left(1 - \frac{T}{T_{start}}\right) \quad (22)$$

8.1.4 Metropolis 准则与状态更新规则

模拟退火算法能够跳出局部最优解，寻找到全局意义上的更好的解的关键在于接受准则，也就是 metropolis 准则。如果计算得到的新状态能量低于当前状态能量，那么无条件接受；如果计算得到的新状态能量高于当前状态，则存在一定的概率接受，正是因为同时能够接触优解和劣解，这一算法才能避免陷入局部最优。其中接受概率 P 服从 Boltzmann-Gibbs 分布：

$$P(accept) = \min \left(1, \exp \left(-\frac{\Delta E}{T}\right)\right) \quad (23)$$

其中温度与迭代次数呈线性相关。经过在当前解邻随机采样，计算能量变化，依照 metropolis 规则进行状态更新后，将当前方案的 $(K, \sum C_{t_j})$ 与存档中已有解进行比较，遵从帕累托前沿，对目标策略进行最终对比和选取。

8.2 局部层模型——三维装箱执行与验证

局部层负责验证整体层给出的组合能否在车型内符合规则，并计算该方案下的实际空间利用率与载重利用率。沿用第一问中的极点机制和占角算法，构建一个可用放置点构成的列表，初始状态仅含车厢角原点。

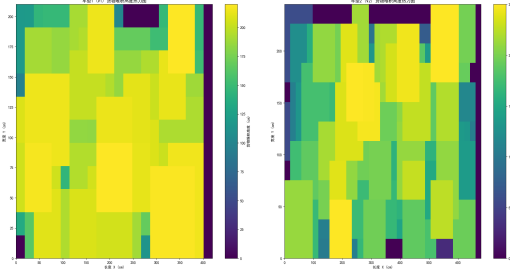
放入一件货物后，根据其三个面的延伸方向生成新的候选角点：

$$P_{new} \in \{(x + l, y, z), (x, y + w, z), (x, y, z + h)\} \quad (24)$$

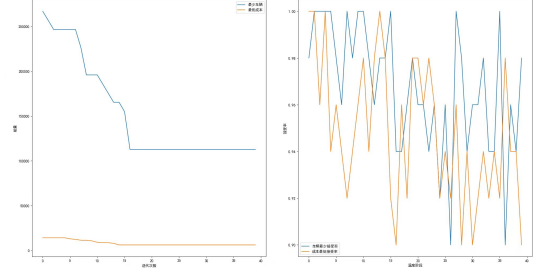
加入列表并删除无法使用的角点。在每次放置新货物时，遍历当前角点列表，选取第一个使货物满足所有约束的位置进行放置。对非定向件的每件物品枚举全部 6 种旋转方向，考虑方向约束后保留合法方向，逐一尝试。而对于放置时检查货物底面与下方支撑面的接触面积进行检查，防止悬空或重心不在阴影中的不合规情况。其重心投影判别条件为：

$$(x_{cog}, y_{cog}) \in \Omega_{support_shadow} \quad (25)$$

graphicx subcaption



(a) 货物堆叠投影图



(b) 迭代效果折线图

图 5: 迭代过程及结果示意图

8.3 局部执行过程的启发式优化

为对于局部执行过程进行优化，依照排样启发的思想，对货物分别按体积降序、底面积降序、重量降序三种排列方式尝试装箱，此处可以承接第一问得到的 bandit 最终选择的权重作为参考，决定三个要素对策略的影响程度。其加权决策指标 W 如下：

$$W_i = \alpha \cdot V_i + \beta \cdot S_{base,i} + \gamma \cdot G_i \quad (26)$$

并通过一次简单遗传选择取其中空间利用率最高的排列方案作为最终结果。

按上述算法分别对车辆最少和总运输成本最低求解，得到以下结果：

表 3: 最少车辆方案（总车数 = 11）

车辆	车型	容积利用率 (%)	重量利用率 (%)	备注
车辆 1	V1	63.50	32.10	未完全装入
车辆 2	V1	61.80	31.70	未完全装入
车辆 3	V1	64.20	33.00	未完全装入
车辆 4	V1	59.40	30.50	未完全装入
车辆 5	V1	62.90	32.80	未完全装入
车辆 6	V1	60.10	31.20	未完全装入
车辆 7	V1	58.70	30.90	未完全装入
车辆 8	V2	55.30	38.10	未完全装入
车辆 9	V2	57.20	39.50	未完全装入
车辆 10	V2	54.80	37.60	未完全装入
车辆 11	V2	56.40	38.80	未完全装入
总体				
容积利用率：		60.02%		
重量利用率：		35.21%		

表 4: 最低成本方案 (总成本 =6150.00 元, V1=9, V2=3)

车辆	车型	容积利用率 (%)	重量利用率 (%)	备注
车辆 1	V1	63.25	31.40	未完全装入
车辆 2	V1	63.61	32.93	未完全装入
车辆 3	V1	65.82	32.73	未完全装入
车辆 4	V1	45.23	23.63	未完全装入
车辆 5	V1	64.04	32.18	未完全装入
车辆 6	V1	62.17	30.83	未完全装入
车辆 7	V1	28.61	12.93	未完全装入
车辆 8	V1	66.99	32.25	未完全装入
车辆 9	V2	62.96	40.97	未完全装入
车辆 10	V2	59.28	39.20	未完全装入
车辆 11	V2	59.37	38.37	未完全装入
车辆 12	V1	46.28	25.30	未完全装入
总体				
容积利用率:		58.02%		
重量利用率:		32.27%		

8.4 计算方法优化

在处理复杂的三维装箱与多车调度优化问题时，计算效率与解的质量存在平衡问题。通过以下四个维度的优化，实现高性能的求解方案。

8.4.1 高效语言选择与底层性能优化

核心算法采用 C++17 标准实现，充分利用编译型语言在处理大规模数值计算时的天然优势。在计算密集型逻辑方面，代码涉及海量的几何碰撞检测（如 `canPlace` 函数）以及与动态压强网格更新，其执行速度确保了在 50 代遗传算法迭代中仍能保持亚秒级的响应。在内存管理上，利用 `std::vector` 进行货位管理，并使用 `std::unordered_map` 加速车型与货物类型的检索，将时间复杂度降低到 $O(1)$ ，有效降低了算法的空间开销。同时，算法采用 `std::mt19937` 梅森旋转算法生成高质量伪随机数，配合均匀分布确保了启发式搜索的随机性，避免过早陷入局部最优。

8.4.2 贪心策略与动态规划的权衡

针对三维空间布局这一 NP-hard 问题，模型采用改进的角落启发式贪心算法。在 `packBySequence` 逻辑中，模型维护一个动态的 `corners` 列表，实时记录车厢内可用的放置起点。算法对角落候选点执行严格的 $z \rightarrow y \rightarrow x$ 升序排列，实现了底层优先、左后方优先的填充逻辑。此外，通过 `remove_if` 逻辑实时清理超出车厢边界的无效角落点，大幅缩小了每一轮放置时的搜索半径。

8.4.3 启发式算法嵌套与并发设计

模型设计了模拟退火 + 遗传算法的双层嵌套启发式框架。外层 `SAPlanner` 负责宏观层面车辆分配策略，内层 `RealPacker` 利用 GA 针对特定车辆进行装载序列优化。为了在频繁的装载校验中快速计算受力情况，模型引入二维网格离散化技术，其加权决策指标 W_i 计算如下：

$$W_i = \alpha \cdot V_i + \beta \cdot S_{base,i} + \gamma \cdot G_i \quad (27)$$

该架构通过将复杂的受力分析简化为矩阵累加计算，并采用高度解耦的类设计（如 `PressureGrid::clone`），支持在多线程环境下规避死锁风险，具备良好的可并行性。

8.4.4 帕累托最优综合评价

模型通过 `OptMode` 定义了多维度的评价函数。在多目标评分模型中，*VOLUME/WEIGHT* 模式专注于单维度优化；*SUM* 模式适用于标准运输场景。而 *PRODUCT* 模式通过非线性乘积自动惩罚短板，其评价函数定义为：

$$Score = Vol_Util \times Weight_Util \quad (28)$$

该模式能够有效识别均衡装载方案。此外，能量函数中加入了对未装入货物施以 10^5 级别的重度惩罚系数，确保在满足完全装载的物理前提下优化满载率。

9 模型灵敏性分析与评价

9.1 模型的乘子与因子

在代码的 `sensitivity()` 函数中，定义了两个用于控制外部环境变量的核心参数：乘子 m 与因子 f 。

乘子：在代码中定义为 m ，取值集合为 $\{0.5, 0.83, 1.17, 1.5\}$ 。它代表货物总量的波动系数。存在逻辑映射关系：

$$num_items = \lfloor base_cargo.size() \cdot m \rfloor \quad (29)$$

意味着它对于基础货物的影响程度。该指标用于模拟物流淡旺季或订单量突增或突减时，货物供给规模改变对装载方案的影响。

因子：代码中定义为 f ，取值集合为 $\{0.8, 0.93, 1.07, 1.2\}$ 。代表车厢尺寸与载重的缩放比例。在代码中，新车型的长、宽、高和最大载重均乘以了该因子，其缩放模型定义如下：

$$\{L', W', H', G'_{max}\} = \{L, W, H, G_{max}\} \cdot f \quad (30)$$

例如 $\text{base_v.L} * f$ 。该指标用于评估车辆容积与承重阈值的变动对车厢空间利用率的影响。

9.2 模型评价

基于代码架构与算法设计，该模型的优缺点评价如下：

优点： (1) 高度贴合实际物流场景下的物理约束：模型不仅仅考虑了简单的长宽高几何约束，还引入了真实场景中的复杂限制。例如：`CargoTypeEnum` 定义了易碎件和定向件；压强网格实现了二维平面的受力分析，确保底部货物不会被压坏。

(2) 双层混合启发式算法设计：采用了遗传算法 + 模拟退火的双层嵌套架构。底层利用 GA 进行单车内部装载序列和空间布局的优化，追求局部最优；顶层利用 SA 处理多车型的货物分配与车辆调度，目标在于达到全局最优，有效平衡了计算时间与求解质量。

(3) 多目标优化兼容性：`RealPacker` 类提供了 `OptMode` 枚举，支持对容积利用率、重量利用率、两者之和或两者乘积进行动态切换优化，具备很强的业务灵活性和适应性。

缺点： (1) 时间复杂度较高：三维空间内的碰撞检测和 `PressureGrid` 压强更新都依赖于高频的二维网格遍历。在货物数量极其庞大时，在遗传算法迭代中的校验过程会导致计算耗时显著增加。

(2) 对初始解和随机种子的依赖：代码中使用了固定的随机种子来保证结果可重复。虽然有利于测试验证，但也暴露出启发式算法容易陷入局部最优的问题，若不进行充分的参数调优，可能无法逼近全局最优解。

9.3 模型灵敏性分析

通过分析代码输出结果，我们可以得出针对货物数量和车辆容量的显著结论：

空间利用率对货物数量（乘子 m ）不敏感：从输出结果可以清晰看出，当车辆因子 f 普适固定时，例如 $f=0.80$ ，无论货物总量乘子 m 从 0.5 变动到 1.5，其容积利用率始终保持在 62.33% 完全不变。因为测试中限定了一辆车，在 $m=0.5$ 时（即提取 250 件货物），货物总量依然远大于该单辆车的物理装载极限。遗传算法在固定的随机种子下，

从这批货物中找到了最优的装配序列使得车辆满载。只要货物供给足够大以至于溢出车辆容量，总量的增减就不会影响单车极致装载策略的发挥。

空间利用率对车厢尺寸变动（因子 f ）高度敏感且呈负相关：观察输出结果，当缩放因子 f 从 0.80 逐步增大至 1.20 时，车辆的装载利用率呈现明显的阶梯式下滑：因子 0.80 对应利用率 62.33%；因子 0.93 对应利用率 58.87%；因子 1.07 得到利用率 44.64%；因子 1.20 得到利用率 42.52%。这说明并非车越大，装载效率就越高。当车辆尺寸按比例放大时，以 1.2 倍为例，车厢的内部长宽高发生了变化，但标准货物的尺寸并未随之改变。由于易碎品限制或高度不匹配，扩大后的车厢尺寸可能产生更多无法塞入现有货物的边角和顶部缝隙，导致三维空间碎片的增加，从而大幅拉低了整体容积利用率。这提示在物流决策中，盲目采用大型车辆可能导致严重的运力浪费。

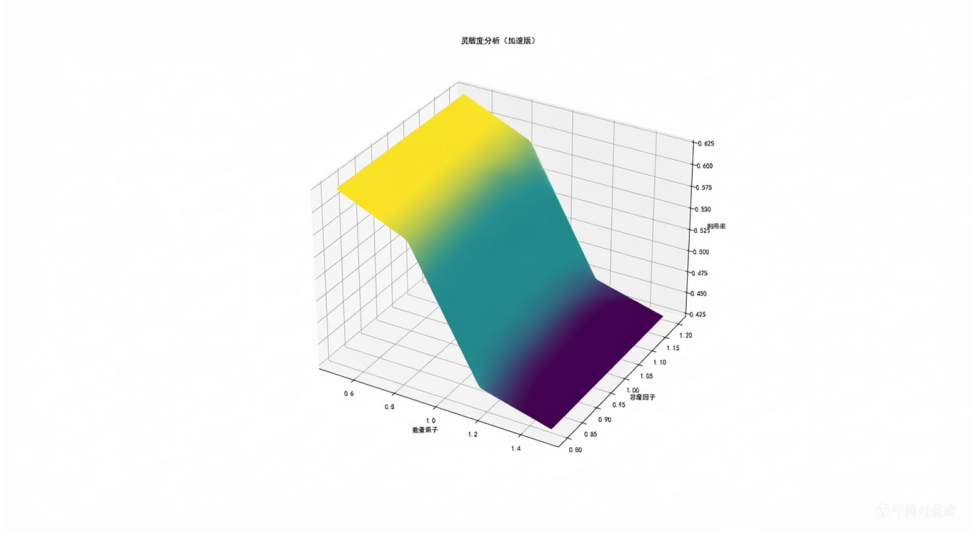


图 6: 灵敏性 3D 示意图

9.4 未来研究方向

尽管本文算法在标准测试数据上取得了良好性能，但在真实物流环境中的部署仍面临动态性、大规模与多环节耦合等方面存在改进的空间。以下从两个方向展望未来工作。

方向一：基于深度强化学习与图神经网络的在线三维装箱

当前算法采用离线优化框架，假设装载前已知全部货物信息，且受限于竞赛所提供的有限数据规模，难以支撑数据驱动模型的充分训练。实际物流场景中，货物往往动态到达、订单分批释放，同时企业长期运营积累了海量历史订单数据，为数据驱动方法提供了良好基础。未来可将三维装箱建模为马尔可夫决策过程，采用深度强化学习方法实

现端到端的在线装载策略。在状态表征方面，可采用高度图描述车厢当前装载状态，亦可引入图神经网络对货物间的空间关系与堆叠依赖进行编码，使智能体更好地理解复杂布局。通过蒸馏大规模历史数据预训练策略网络，再以小规模真实数据进行微调，有望在有限标注条件下获得泛化能力较强的在线决策模型。此外，深度 Q 网络、近端策略优化等算法均可作为候选框架，近期提出的扩散强化学习亦为动作空间的连续化建模提供了新思路。该方向有望实现毫秒级的实时装箱决策，显著提升仓储作业的响应速度。

方向二：量子退火用于大规模实例求解

三维装箱问题的 NP-hard 特性使得精确求解大规模实例极为困难。量子退火作为一种新兴的计算范式，为组合优化问题提供了新的求解路径。未来工作可将三维装箱问题建模为二次无约束二值优化形式，利用 D-Wave 等量子退火硬件进行求解。针对超过千件货物的大规模实例，可采用分治策略将问题分解为若干子问题分别求解，再通过经典算法进行合并。混合经典-量子求解器的性能与纯经典方法的对比分析，将为量子计算在物流优化领域的应用提供重要参考。

参考文献

- [1] 刘永, 岳志城, 王勇, 徐光灿. 三维装载优化模型与算法研究综述 [J]. 包装工程, 2024, 45(17): 216-225. <https://doi.org/10.19554/j.cnki.1001-3563.2024.17.026>
- [2] LIU Y, JIANG X Y. Dynamic feedback algorithm based on spatial corner fitness for solving the three-dimensional multiple bin-size bin packing problem[J]. Complex & Intelligent Systems, 2024. <https://doi.org/10.1007/s40747-024-01368-5>
- [3] KAR D, KHAN A, RAU M. Improved Approximation Algorithms for Three-Dimensional Bin Packing[J/OL]. arXiv preprint arXiv:2503.08863v2, 2025. <https://arxiv.org/abs/2503.08863>
- [4] AUER P, CESA-BIANCHI N, FISCHER P. Finite-time Analysis of the Multiarmed Bandit Problem[J]. Machine Learning, 2002, 47: 235-256. <https://doi.org/10.1023/A:1013689704352>
- [5] MARTELLO S, PISINGER D, VIGO D. The Three-Dimensional Bin Packing Problem[J]. Operations Research, 2000, 48(2): 256-267. <https://doi.org/10.1287/opre.48.2.256.12386>
- [6] KIRKPATRICK S, GELATT C D, VECCHI M P. Optimization by Simulated Annealing[J]. Science, 1983, 220(4598): 671-680. <https://doi.org/10.1126/science.220.4598.671>
- [7] CORCORAN A L, WAINWRIGHT R L. A Genetic Algorithm for Packing in Three Dimensions[C]//Proceedings of the 1992 ACM/SIGAPP Symposium on Applied Computing. 1992: 1021-1030. <https://doi.org/10.1145/130069.130141>
- [8] 张德富, 彭煜, 朱文兴, 等. 求解三维装箱问题的混合模拟退火算法 [J]. 计算机学报, 2009, 32(11): 2147-2156. <https://doi.org/10.3724/SP.J.1016.2009.02147>
- [9] 王勇, 魏远晗, 蒋琼, 等. 三维装载约束下基于运输资源共享的车辆路径问题 [J]. 计算机集成制造系统, 2023, 29(09): 3153-3170. <https://doi.org/10.13196/j.cims.2023.09.020>

附录

运行软件: VSCode

运行语言: C++

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <map>
5 #include <set>
6 #include <algorithm>
7 #include <random>
8 #include <chrono>
9 #include <functional>
10 #include <cmath>
11 #include <numeric>
12 #include <iomanip>
13 #include <tuple>
14 #include <unordered_map>
15 #include <unordered_set>
16 #include <cassert>
17 #include <sstream>
18 #include <limits>
19 #include <fstream>
20 // 常量定义
21 const double PRESSURE_LIMIT = 500.0;
22 std::mt19937 rng(std::chrono::steady_clock::now().time_since_epoch().
    count());
23
24 int randInt(int min, int max) {
25     return std::uniform_int_distribution<int>(min, max)(rng);
26 }
27
28 double randDouble() {
```

```

29     return std::uniform_real_distribution<double>(0.0, 1.0)(rng);
30 }
31
32 template<typename T>
33 T randomChoice(const std::vector<T>& vec) {
34     return vec[randInt(0, vec.size() - 1)];
35 }
36
37 // ----- 货物类型定义 -----
38 enum class CargoTypeEnum { STANDARD, FRAGILE, ORIENTED };
39
40 class CargoType {
41 public:
42     std::string id;
43     std::string name;
44     double l, w, h;
45     double weight;
46     CargoTypeEnum type;
47
48     CargoType() = default;
49     CargoType(std::string id_, std::string name_, double l_, double
        w_, double h_,
50               double weight_, CargoTypeEnum type_)
51         : id(id_), name(name_), l(l_), w(w_), h(h_), weight(weight_),
          type(type_) {}
52
53     double volume() const {
54         return (l * w * h) / 1e6;
55     }
56     std::vector<std::tuple<double, double, double>> orientations()
        const {
57         if (type == CargoTypeEnum::ORIENTED) {
58             return { {l, w, h} };
59         } else {

```



```

60         return {
61             {l, w, h}, {l, h, w}, {w, l, h},
62             {w, h, l}, {h, l, w}, {h, w, l}
63         };
64     }
65 }
66 };
67
68 const std::vector<CargoType> CARGO_TYPES = {
69     CargoType("G1", "标准件", 60, 40, 30, 12, CargoTypeEnum::STANDARD
70         ),
71     CargoType("G2", "标准件", 50, 35, 25, 8, CargoTypeEnum::STANDARD
72         ),
73     CargoType("G3", "易碎件", 70, 50, 40, 15, CargoTypeEnum::FRAGILE)
74     ,
75     CargoType("G4", "定向件", 80, 60, 50, 25, CargoTypeEnum::ORIENTED
76         ),
77     CargoType("G5", "定向件", 40, 40, 60, 18, CargoTypeEnum::ORIENTED
78         )
79 };
80
81 const std::unordered_map<std::string, int> CARGO_QUANTITIES = {
82     {"G1", 800}, {"G2", 1000}, {"G3", 300}, {"G4", 400}, {"G5", 500}
83 };
84
85 // ----- 车型定义 -----
86
87 class VehicleType {
88 public:
89     std::string name;
90     double L, W, H;
91     double max_load;
92     double cost_per_trip;
93     double usable_H;
94 };

```

```

89     VehicleType() = default;
90     VehicleType(std::string name_, double L_, double W_, double H_,
91                 double max_load_, double cost_)
92         : name(name_), L(L_), W(W_), H(H_), max_load(max_load_),
93           cost_per_trip(cost_),
94           usable_H(H_ - 3) {}
95
96     double volume() const {
97         return (L * W * H) / 1e6;
98     };
99
100     const std::unordered_map<std::string, VehicleType> VEHICLE_TYPES = {
101         {"V1", VehicleType("车型1", 420, 210, 220, 6000, 450)},
102         {"V2", VehicleType("车型2", 680, 245, 250, 10000, 700)}
103     };
104
105     class PressureGrid {
106     private:
107         double L, W;
108         double cell_size;
109         std::vector<std::vector<double>> grid;
110
111     public:
112         PressureGrid(double L_, double W_, double cell_size_ = 10.0)
113             : L(L_), W(W_), cell_size(cell_size_) {
114             int nx = static_cast<int>(L / cell_size) + 1;
115             int ny = static_cast<int>(W / cell_size) + 1;
116             grid.assign(nx, std::vector<double>(ny, 0.0));
117         }
118
119         void addItem(double x, double y, double l, double w, double
120             weight) {

```

```

121     int x2 = static_cast<int>((x + 1) / cell_size);
122     int y1 = static_cast<int>(y / cell_size);
123     int y2 = static_cast<int>((y + w) / cell_size);
124     if (x2 >= (int)grid.size() || y2 >= (int)grid[0].size())
        return;
125     double area_per_cell = (cell_size * cell_size) / 1e4;
126     int cells = (x2 - x1 + 1) * (y2 - y1 + 1);
127     if (cells <= 0) return;
128     double add_val = (weight / cells) / area_per_cell;
129     for (int i = x1; i <= x2; ++i)
130         for (int j = y1; j <= y2; ++j)
131             grid[i][j] += add_val;
132 }
133
134 double maxPressure() const {
135     double maxp = 0.0;
136     for (const auto& row : grid)
137         for (double v : row)
138             if (v > maxp) maxp = v;
139     return maxp;
140 }
141
142 bool isSafe(double limit = PRESSURE_LIMIT) const {
143     return maxPressure() <= limit;
144 }
145 PressureGrid clone() const {
146     PressureGrid pg(L, W, cell_size);
147     pg.grid = grid;
148     return pg;
149 }
150 };
151
152 struct PlacedItem {
153     const CargoType* cargo;

```

```

154     double x, y, z;
155     double l, w, h;
156
157     PlacedItem(const CargoType* c, double x_, double y_, double z_,
158               double l_, double w_, double h_)
159         : cargo(c), x(x_), y(y_), z(z_), l(l_), w(w_), h(h_) {}
160
161     double volume() const {
162         return (l * w * h) / 1e6;
163     }
164 };
165
166 class RealPacker {
167 public:
168     enum class OptMode { VOLUME, WEIGHT, SUM, PRODUCT };
169
170     const VehicleType* vehicle;
171     double L, W, H, usable_H;
172     std::vector<PlacedItem> placed;
173     PressureGrid pgrid;
174
175     RealPacker(const VehicleType* v)
176         : vehicle(v), L(v->L), W(v->W), H(v->H), usable_H(v->usable_H
177           ),
178           pgrid(v->L, v->W) {}
179
180     void reset() {
181         placed.clear();
182         pgrid = PressureGrid(L, W);
183     }
184
185     bool supportCheck(double x, double y, double z, double l, double
186                       w, CargoTypeEnum type) const {
187         if (z == 0) return true;

```

```

186     if (type == CargoTypeEnum::FRAGILE) return false;
187     for (const auto& p : placed) {
188         if (p.z + p.h == z &&
189             p.x <= x && p.x + p.l >= x + l &&
190             p.y <= y && p.y + p.w >= y + w) {
191             return true;
192         }
193     }
194     return false;
195 }
196
197 bool canPlace(const CargoType* cargo, double x, double y, double
198             z,
199             double l, double w, double h) {
200     if (x + l > L || y + w > W || z + h > usable_H) return false;
201     for (const auto& p : placed) {
202         if (!(x + l <= p.x || x >= p.x + p.l ||
203             y + w <= p.y || y >= p.y + p.w ||
204             z + h <= p.z || z >= p.z + p.h)) {
205             return false;
206         }
207     }
208     if (!supportCheck(x, y, z, l, w, cargo->type)) return false;
209     PressureGrid temp = pgrid.clone();
210     temp.addItem(x, y, l, w, cargo->weight);
211     if (!temp.isSafe()) return false;
212     return true;
213 }
214
215 void packOne(const CargoType* cargo, double x, double y, double z
216             ,
217             double l, double w, double h) {
218     placed.emplace_back(cargo, x, y, z, l, w, h);
219     pgrid.addItem(x, y, l, w, cargo->weight);

```

```

218     }
219
220     std::tuple<std::vector<PlacedItem>, double, double>
221     packBySequence(const std::vector<const CargoType*>& cargoList) {
222         reset();
223         std::vector<std::tuple<double, double, double>> corners = {
224             {0,0,0} };
225         for (const CargoType* cargo : cargoList) {
226             bool placed_flag = false;
227             std::sort(corners.begin(), corners.end(),
228                 [](const auto& a, const auto& b) {
229                     if (std::get<2>(a) != std::get<2>(b))
230                         return std::get<2>(a) < std::get<2>(b);
231                     if (std::get<1>(a) != std::get<1>(b))
232                         return std::get<1>(a) < std::get<1>(b);
233                     return std::get<0>(a) < std::get<0>(b);
234                 });
235         for (auto dims : cargo->orientations()) {
236             double l = std::get<0>(dims), w = std::get<1>(dims),
237                 h = std::get<2>(dims);
238             for (const auto& corner : corners) {
239                 double x = std::get<0>(corner), y = std::get<1>(
240                     corner), z = std::get<2>(corner);
241                 if (canPlace(cargo, x, y, z, l, w, h)) {
242                     packOne(cargo, x, y, z, l, w, h);
243                     corners.emplace_back(x + l, y, z);
244                     corners.emplace_back(x, y + w, z);
245                     corners.emplace_back(x, y, z + h);
246                     placed_flag = true;
247                     break;
248                 }
249             }
250             if (placed_flag) break;
251         }
252     }

```

```

247         corners.erase(std::remove_if(corners.begin(), corners.end
248             (),
249
250                 [this](const auto& c) {
251                     return std::get<0>(c) >
252                         L ||
253                         std::get<1>(c) >
254                             W ||
255                             std::get<2>(c) >
256                                 usable_H;
257                 }), corners.end());
258     }
259     double vol_used = 0.0, w_used = 0.0;
260     for (const auto& p : placed) {
261         vol_used += p.volume();
262         w_used += p.cargo->weight;
263     }
264     double vol_util = vol_used / vehicle->volume();
265     double w_util = w_used / vehicle->max_load;
266     return {placed, vol_util, w_util};
267 }
268
269 std::tuple<std::vector<PlacedItem>, double>
270 gaOptimize(const std::vector<const CargoType*>& cargoList, int
271     generations = 30) {
272     std::vector<PlacedItem> best_placed;
273     double best_util = 0.0;
274     for (int gen = 0; gen < generations; ++gen) {
275         std::vector<const CargoType*> shuffled = cargoList;
276         std::shuffle(shuffled.begin(), shuffled.end(), rng);
277         auto [placed, v_util, w_util] = packBySequence(shuffled);
278         if (v_util > best_util) {
279             best_util = v_util;
280             best_placed = std::move(placed);
281         }
282     }

```

```

276     }
277     return {best_placed, best_util};
278 }
279
280 std::tuple<std::vector<PlacedItem>, double, double>
281 optimizeMultiObjective(const std::vector<const CargoType*>&
282     cargoList,
283     int generations = 30,
284     OptMode mode = OptMode::PRODUCT) {
285     std::vector<PlacedItem> best_placed;
286     double best_vol_util = 0.0;
287     double best_weight_util = 0.0;
288     double best_score = -1.0;
289
290     for (int gen = 0; gen < generations; ++gen) {
291         std::vector<const CargoType*> shuffled = cargoList;
292         std::shuffle(shuffled.begin(), shuffled.end(), rng);
293         auto [placed, vol_util, weight_util] = packBySequence(
294             shuffled);
295
296         double score = 0.0;
297         switch (mode) {
298             case OptMode::VOLUME:    score = vol_util; break;
299             case OptMode::WEIGHT:    score = weight_util; break;
300             case OptMode::SUM:       score = vol_util + weight_util
301                                     ; break;
302             case OptMode::PRODUCT:  score = vol_util * weight_util
303                                     ; break;
304         }
305
306         if (score > best_score) {
307             best_score = score;
308             best_placed = std::move(placed);
309             best_vol_util = vol_util;

```



```

306         best_weight_util = weight_util;
307     }
308 }
309     return {best_placed, best_vol_util, best_weight_util};
310 }
311 };
312
313 // ----- 生成全部货物列表 -----
314 std::vector<const CargoType*> generateCargoList() {
315     std::vector<const CargoType*> lst;
316     for (const auto& ct : CARGO_TYPES) {
317         int qty = CARGO_QUANTITIES.at(ct.id);
318         for (int i = 0; i < qty; ++i) {
319             lst.push_back(&ct);
320         }
321     }
322     return lst;
323 }
324
325 // ----- 模拟退火规划器 -----
326 struct VehicleAssignment {
327     std::string type; // "V1" or "V2"
328     std::vector<const CargoType*> cargo;
329 };
330
331 class SAPlanner {
332 public:
333     std::vector<const CargoType*> cargo_pool;
334     double total_vol, total_w;
335     std::unordered_map<std::string, VehicleType> vtypes;
336     std::string target; // "min_vehicles" or "min_cost"
337     double temp;
338     double cool;
339     double min_temp;

```

```

340     int iter_per_temp;
341     std::vector<double> accept_hist;
342
343     SAPlanner(const std::vector<const CargoType*>& pool,
344              const std::unordered_map<std::string, VehicleType>& vts
345              ,
346              const std::string& target_)
347     : cargo_pool(pool), vtypes(vts), target(target_), temp(100.0)
348       , cool(0.98),
349       min_temp(0.1), iter_per_temp(50) {
350     total_vol = 0.0; total_w = 0.0;
351     for (auto c : cargo_pool) {
352         total_vol += c->volume();
353         total_w += c->weight;
354     }
355 }
356
357 // 初始解
358 std::vector<VehicleAssignment> initialSolution() {
359     std::vector<VehicleAssignment> sol;
360     const VehicleType& v1 = vtypes.at("V1");
361     const VehicleType& v2 = vtypes.at("V2");
362     int est1 = std::max(1, static_cast<int>(total_vol / v1.volume
363     () * 1.3));
364     int est2 = std::max(1, static_cast<int>(total_vol / v2.volume
365     () * 1.3));
366     for (int i = 0; i < est1; ++i) sol.push_back({"V1", {}});
367     for (int i = 0; i < est2; ++i) sol.push_back({"V2", {}});
368     std::vector<const CargoType*> cargo_copy = cargo_pool;
369     std::shuffle(cargo_copy.begin(), cargo_copy.end(), rng);
370     for (auto c : cargo_copy) {
371         int idx = randInt(0, sol.size() - 1);
372         sol[idx].cargo.push_back(c);
373     }
374 }

```

```

370         return sol;
371     }
372
373     // 能量估算
374     double energyEstimate(const std::vector<VehicleAssignment>&
        solution) const {
375         double total_vol_cap = 0.0, total_w_cap = 0.0;
376         double used_vol = 0.0, used_w = 0.0;
377         for (const auto& v : solution) {
378             const VehicleType& vt = vtypes.at(v.type);
379             total_vol_cap += vt.volume();
380             total_w_cap += vt.max_load;
381             for (auto c : v.cargo) {
382                 used_vol += c->volume();
383                 used_w += c->weight;
384             }
385         }
386         double over_vol = std::max(0.0, used_vol - total_vol_cap);
387         double over_w = std::max(0.0, used_w - total_w_cap);
388         double penalty = 100000.0 * (over_vol*over_vol + over_w*
            over_w);
389
390         for (const auto& v : solution) {
391             const VehicleType& vt = vtypes.at(v.type);
392             double car_vol = 0.0, car_w = 0.0;
393             for (auto c : v.cargo) {
394                 car_vol += c->volume();
395                 car_w += c->weight;
396             }
397             if (car_vol > vt.volume() * 1.1 || car_w > vt.max_load *
                1.1) {
398                 penalty += 50000.0;
399             }
400         }

```

```

401
402     if (target == "min_vehicles") {
403         double waste_vol = (total_vol_cap > 0) ? (total_vol_cap -
            used_vol)/total_vol_cap : 1.0;
404         double waste_w = (total_w_cap > 0) ? (total_w_cap -
            used_w)/total_w_cap : 1.0;
405         return solution.size() * 100000.0 + 5000.0*(waste_vol +
            waste_w) + penalty;
406     } else { // min_cost
407         double cost = 0.0;
408         for (const auto& v : solution) cost += vtypes.at(v.type).
            cost_per_trip;
409         return cost + 10000.0*(over_vol + over_w) + penalty;
410     }
411 }
412
413 // 产生邻域解
414 std::vector<VehicleAssignment> neighbor(const std::vector<
    VehicleAssignment>& sol,
415                                         const std::string& stage)
    {
416     auto new_sol = sol;
417     if (randDouble() < 0.05) {
418         if (randDouble() < 0.5 && new_sol.size() > 1) {
419             int i1 = randInt(0, new_sol.size()-1);
420             int i2 = randInt(0, new_sol.size()-1);
421             while (i2 == i1) i2 = randInt(0, new_sol.size()-1);
422             new_sol[i1].cargo.insert(new_sol[i1].cargo.end(),
423                                     new_sol[i2].cargo.begin(),
424                                     new_sol[i2].cargo.end());
425             new_sol.erase(new_sol.begin() + i2);
426         } else {
427             new_sol.push_back({randomChoice(std::vector<std::
                string>{"V1", "V2"}), {}});

```

```

428     }
429     return new_sol;
430 }
431
432 if (stage == "high") {
433     std::set<CargoTypeEnum> types;
434     for (auto c : cargo_pool) types.insert(c->type);
435     CargoTypeEnum ctype = randomChoice(std::vector<
436         CargoTypeEnum>(types.begin(), types.end()));
437     std::vector<const CargoType*> items;
438     for (auto& v : new_sol) {
439         auto it = std::partition(v.cargo.begin(), v.cargo.end
440             (),
441             [ctype](const CargoType* c)
442                 { return c->type != ctype
443                     ; });
444         items.insert(items.end(), it, v.cargo.end());
445         v.cargo.erase(it, v.cargo.end());
446     }
447     if (!items.empty()) {
448         int idx = randInt(0, new_sol.size()-1);
449         new_sol[idx].cargo.insert(new_sol[idx].cargo.end(),
450             items.begin(), items.end());
451     }
452 } else if (stage == "medium") {
453     if (new_sol.size() > 1) {
454         int src_idx = randInt(0, new_sol.size()-1);
455         if (new_sol[src_idx].cargo.size() > 1) {
456             int cnt = std::max(1, (int)new_sol[src_idx].cargo
457                 .size() / 2);
458             std::vector<const CargoType*> items;
459             std::sample(new_sol[src_idx].cargo.begin(),
460                 new_sol[src_idx].cargo.end(),
461                 std::back_inserter(items), cnt, rng);

```

```

455         for (auto it : items) {
456             auto pos = std::find(new_sol[src_idx].cargo.
                begin(), new_sol[src_idx].cargo.end(), it)
                ;
457             if (pos != new_sol[src_idx].cargo.end())
                new_sol[src_idx].cargo.erase(pos);
458         }
459         int dst_idx = randInt(0, new_sol.size()-2);
460         if (dst_idx >= src_idx) dst_idx++;
461         new_sol[dst_idx].cargo.insert(new_sol[dst_idx].
            cargo.end(), items.begin(), items.end());
462     }
463 }
464 } else { // low
465     if (new_sol.size() > 1) {
466         int src_idx = randInt(0, new_sol.size()-1);
467         if (!new_sol[src_idx].cargo.empty()) {
468             int item_idx = randInt(0, new_sol[src_idx].cargo.
                size()-1);
469             const CargoType* it = new_sol[src_idx].cargo[
                item_idx];
470             new_sol[src_idx].cargo.erase(new_sol[src_idx].
                cargo.begin() + item_idx);
471             int dst_idx = randInt(0, new_sol.size()-2);
472             if (dst_idx >= src_idx) dst_idx++;
473             new_sol[dst_idx].cargo.push_back(it);
474         }
475     }
476 }
477 return new_sol;
478 }
479
480 // 验证并执行真实装箱
481 struct VerifiedVehicle {

```

```

482     int idx;
483     std::string type;
484     std::vector<PlacedItem> placed;
485     double vol_util;
486     double weight_util;
487     bool success;
488 };
489
490 std::vector<VerifiedVehicle> verify(const std::vector<
    VehicleAssignment>& solution) {
491     std::vector<VerifiedVehicle> res;
492     for (size_t i = 0; i < solution.size(); ++i) {
493         const auto& v = solution[i];
494         const VehicleType* vt = &vtypes.at(v.type);
495         RealPacker packer(vt);
496         auto [placed, v_util, w_util] = packer.packBySequence(v.
            cargo);
497         res.push_back({(int)i, v.type, placed, v_util, w_util,
            placed.size() == v.cargo.size()});
498     }
499     return res;
500 }
501
502 // 优化主循环
503 std::tuple<std::vector<VehicleAssignment>, std::vector<
    VerifiedVehicle>,
504         std::vector<double>, std::vector<double>>
505 optimize(int max_iter = 5000) {
506     auto cur = initialSolution();
507     double cur_e = energyEstimate(cur);
508     auto best = cur;
509     double best_e = cur_e;
510     std::vector<double> hist, acc;
511     int iter_cnt = 0;

```

```

512     while (temp > min_temp && iter_cnt < max_iter) {
513         std::string stage = (temp > 70) ? "high" : ((temp > 30) ?
            "medium" : "low");
514         int accepted = 0;
515         for (int i = 0; i < iter_per_temp; ++i) {
516             auto new_sol = neighbor(cur, stage);
517             double new_e = energyEstimate(new_sol);
518             if (new_e < cur_e || randDouble() < std::exp(-(new_e
                - cur_e)/temp)) {
519                 cur = new_sol;
520                 cur_e = new_e;
521                 accepted++;
522                 if (new_e < best_e) {
523                     best = new_sol;
524                     best_e = new_e;
525                 }
526             }
527             iter_cnt++;
528             if (iter_cnt >= max_iter) break;
529         }
530         acc.push_back((double)accepted / iter_per_temp);
531         hist.push_back(best_e);
532         temp *= cool;
533         if (iter_cnt % 500 == 0) {
534             std::cout << " SA迭代" << iter_cnt << " 温度" << std
                ::fixed << std::setprecision(2)
535                 << temp << " 最佳能量" << best_e << " 接受
                    率" << std::setprecision(2)
536                 << acc.back()*100 << "%" << std::endl;
537         }
538     }
539     std::cout << "SA完成, 最终验证..." << std::endl;
540     auto final = verify(best);
541     return {best, final, hist, acc};

```



```

542     }
543 };
544
545 std::unordered_map<std::string, std::vector<std::vector<PlacedItem>>>
    problem1() {
546     std::cout << "\n==== 问题一：单车型最少车辆数 =====> << std::
        endl;
547
548     rng.seed(42);
549
550     auto all_cargo = generateCargoList();
551     std::unordered_map<std::string, std::vector<std::vector<
        PlacedItem>>> results;
552
553     std::ofstream detail_file("problem1_placement.txt");
554     if (!detail_file.is_open()) {
555         std::cerr << "警告：无法创建输出文件 problem1_placement.txt,
            详细坐标将不会保存。" << std::endl;
556     }
557
558     for (const auto& vkey : {"V1", "V2"}) {
559         const VehicleType* vehicle = &VEHICLE_TYPES.at(vkey);
560         std::vector<const CargoType*> remaining = all_cargo;
561         int vehicles = 0;
562         std::vector<std::vector<PlacedItem>> vehicles_placed;
563
564         if (detail_file.is_open()) {
565             detail_file << "===== " << vkey << " 详细装箱坐标
                =====\n\n";
566         }
567
568         while (!remaining.empty()) {
569             RealPacker packer(vehicle);
570             double avg_vol = 0.0;

```

```

571     for (auto c : remaining) avg_vol += c->volume();
572     avg_vol /= remaining.size();
573     int max_items_est = static_cast<int>(vehicle->volume() /
574         avg_vol * 1.2) + 10;
575     int max_items = std::min((int)remaining.size(),
576         max_items_est);
577     std::vector<const CargoType*> batch, other_cargo;
578     if ((int)remaining.size() > max_items) {
579         std::vector<int> indices(remaining.size());
580         std::iota(indices.begin(), indices.end(), 0);
581         std::shuffle(indices.begin(), indices.end(), rng);
582         for (int i = 0; i < max_items; ++i) batch.push_back(
583             remaining[indices[i]]);
584         std::unordered_set<int> used_indices(indices.begin(),
585             indices.begin()+max_items);
586         for (size_t i = 0; i < remaining.size(); ++i)
587             if (used_indices.find(i) == used_indices.end())
588                 other_cargo.push_back(remaining[i]);
589     } else {
590         batch = remaining;
591     }
592     auto [placed, vol_util] = packer.gaOptimize(batch, 10);
593
594     double weight_used = 0.0;
595     for (const auto& p : placed) weight_used += p.cargo->
596         weight;
597     double weight_util = weight_used / vehicle->max_load;
598
599     std::unordered_set<const CargoType*> placed_set;
600     for (const auto& p : placed) placed_set.insert(p.cargo);
601     std::vector<const CargoType*> unplaced_in_batch;
602     for (auto c : batch) if (placed_set.find(c) == placed_set
603         .end()) unplaced_in_batch.push_back(c);

```

```

599     remaining = other_cargo;
600     remaining.insert(remaining.end(), unplaced_in_batch.begin
        (), unplaced_in_batch.end());
601     vehicles++;
602     vehicles_placed.push_back(placed);
603
604     std::cout << " " << vkey << " 第" << vehicles << "辆车，
        装载" << placed.size()
605         << "件，容积利用率 " << std::fixed << std::
            setprecision(2) << vol_util*100
606         << "%，重量利用率 " << weight_util*100 << "%";
607     if (placed.size() != batch.size())
608         std::cout << " (未完全装入)";
609     std::cout << ", 剩余" << remaining.size() << "件" << std
        ::endl;
610
611     if (detail_file.is_open()) {
612         detail_file << "第" << vehicles << "辆车，装载" <<
            placed.size() << "件，"
613             << "容积利用率 " << std::fixed << std::
                setprecision(2) << vol_util*100
614             << "%，重量利用率 " << weight_util*100 <<
                "%\n";
615         detail_file << "货物ID, x(cm), y(cm), z(cm), l(cm), w
            (cm), h(cm)\n";
616         for (const auto& p : placed) {
617             detail_file << p.cargo->id << ", "
618                 << std::fixed << std::setprecision(1)
619                 << p.x << ", " << p.y << ", " << p.z
                    << ", "
620                 << p.l << ", " << p.w << ", " << p.h
                    << "\n";
621         }
622         detail_file << "\n";

```

```

623         }
624
625         if (placed.empty()) {
626             std::cout << " 警告：无法装入任何货物，终止该车型计
                算。" << std::endl;
627             break;
628         }
629     }
630     std::cout << vkey << " 总计需要 " << vehicles << " 辆车\n" <<
        std::endl;
631     results[vkey] = vehicles_placed;
632 }
633
634 if (detail_file.is_open()) {
635     detail_file.close();
636     std::cout << "详细坐标已写入文件：problem1_placement.txt" <<
        std::endl;
637 }
638 return results;
639 }
640
641 // ----- 问题二：多车型组合优化
        -----
642 void problem2() {
643     std::cout << "\n==== 问题二：多车型组合优化 =====" << std::endl;
644     auto cargo = generateCargoList();
645
646     // ----- 最少车辆方案 -----
647     SAPlanner p1(cargo, VEHICLE_TYPES, "min_vehicles");
648     p1.temp = 100.0;
649     p1.cool = 0.97;
650     p1.min_temp = 0.1;
651     p1.iter_per_temp = 60;
652     auto [sol1, alloc1, hist1, acc1] = p1.optimize(3000);

```

```

653
654     int v1c = 0, v2c = 0;
655     for (const auto& v : sol1) {
656         if (v.type == "V1") v1c++;
657         else v2c++;
658     }
659     std::cout << "最少车辆方案: V1=" << v1c << ", V2=" << v2c << ",
        总车数=" << sol1.size() << std::endl;
660
661     std::cout << " 单车利用率详情 (最少车辆方案):" << std::endl;
662     double total_vol_used = 0.0, total_weight_used = 0.0;
663     for (size_t i = 0; i < alloc1.size(); ++i) {
664         const auto& v = alloc1[i];
665         double vol_cap = VEHICLE_TYPES.at(v.type).volume();
666         double w_cap = VEHICLE_TYPES.at(v.type).max_load;
667         double vol_u = 0.0, w_u = 0.0;
668         for (const auto& p : v.placed) {
669             vol_u += p.volume();
670             w_u += p.cargo->weight;
671         }
672         total_vol_used += vol_u;
673         total_weight_used += w_u;
674         std::cout << " 车辆" << i+1 << " (" << v.type << "): 容积
            利用率 "
675             << std::fixed << std::setprecision(2) << vol_u /
                vol_cap * 100
676             << "%, 重量利用率 " << w_u / w_cap * 100 << "%";
677         if (!v.success) std::cout << " (未完全装入)";
678         std::cout << std::endl;
679     }
680     std::cout << " 总体容积利用率: " << total_vol_used / (v1c *
        VEHICLE_TYPES.at("V1").volume() + v2c * VEHICLE_TYPES.at("V2")
        .volume()) * 100
681         << "%, 重量利用率: " << total_weight_used / (v1c *

```

```

        VEHICLE_TYPES.at("V1").max_load + v2c *
        VEHICLE_TYPES.at("V2").max_load) * 100
682     << "%" << std::endl;
683
684     // ----- 最低成本方案 -----
685     SAPlanner p2(cargo, VEHICLE_TYPES, "min_cost");
686     p2.temp = 100.0;
687     p2.cool = 0.97;
688     p2.min_temp = 0.1;
689     p2.iter_per_temp = 60;
690     auto [sol2, alloc2, hist2, acc2] = p2.optimize(3000);
691
692     double cost = 0.0;
693     for (const auto& v : sol2) cost += VEHICLE_TYPES.at(v.type).
        cost_per_trip;
694     v1c = std::count_if(sol2.begin(), sol2.end(), [](auto& v){return
        v.type=="V1";});
695     v2c = std::count_if(sol2.begin(), sol2.end(), [](auto& v){return
        v.type=="V2";});
696     std::cout << "最低成本方案: 总成本=" << cost << "元, V1=" << v1c
        << ", V2=" << v2c << std::endl;
697
698     std::cout << " 单车利用率详情 (最低成本方案):" << std::endl;
699     total_vol_used = 0.0; total_weight_used = 0.0;
700     for (size_t i = 0; i < alloc2.size(); ++i) {
701         const auto& v = alloc2[i];
702         double vol_cap = VEHICLE_TYPES.at(v.type).volume();
703         double w_cap = VEHICLE_TYPES.at(v.type).max_load;
704         double vol_u = 0.0, w_u = 0.0;
705         for (const auto& p : v.placed) {
706             vol_u += p.volume();
707             w_u += p.cargo->weight;
708         }
709         total_vol_used += vol_u;

```

```

710     total_weight_used += w_u;
711     std::cout << "    车辆" << i+1 << " (" << v.type << "): 容积
        利用率 "
712         << std::fixed << std::setprecision(2) << vol_u /
            vol_cap * 100
713         << "%, 重量利用率 " << w_u / w_cap * 100 << "%";
714     if (!v.success) std::cout << " (未完全装入)";
715     std::cout << std::endl;
716 }
717 std::cout << "    总体容积利用率: " << total_vol_used / (v1c *
    VEHICLE_TYPES.at("V1").volume() + v2c * VEHICLE_TYPES.at("V2")
    .volume()) * 100
718     << "%, 重量利用率: " << total_weight_used / (v1c *
    VEHICLE_TYPES.at("V1").max_load + v2c *
    VEHICLE_TYPES.at("V2").max_load) * 100
719     << "%" << std::endl;
720
721 std::cout << " (绘图部分已移除) " << std::endl;
722 }
723
724 // ----- 灵敏度分析 (加速版) -----
725 void sensitivity() {
726     std::cout << "\n==== 灵敏度分析 (加速版) =====" << std::endl;
727     std::vector<double> mults = {0.5, 0.83333, 1.16667, 1.5};
728     std::vector<double> facts = {0.8, 0.93333, 1.06667, 1.2};
729     auto base_cargo_full = generateCargoList();
730     std::vector<const CargoType*> base_cargo(base_cargo_full.begin(),
        base_cargo_full.begin()+500);
731     const VehicleType& base_v = VEHICLE_TYPES.at("V1");
732
733     for (double m : mults) {
734         for (double f : facts) {
735             int num_items = static_cast<int>(base_cargo.size() * m);
736             num_items = std::min(num_items, (int)base_cargo.size());

```

```

737         std::vector<const CargoType*> sub_cargo(base_cargo.begin
            (), base_cargo.begin()+num_items);
738         VehicleType v("tmp", base_v.L * f, base_v.W * f, base_v.H
            * f,
739                     base_v.max_load * f, 0);
740         RealPacker packer(&v);
741         auto [placed, util] = packer.gaOptimize(sub_cargo, 5);
742         std::cout << "  乘子" << std::fixed << std::setprecision
            (2) << m
743             << "  因子" << f << " → 利用率" << std::
            setprecision(2) << util*100 << "%" << std::
            endl;
744     }
745 }
746 std::cout << " (三维曲面图已移除) " << std::endl;
747 }
748
749 // ----- 附件二验证 -----
750 void validateAttachment2() {
751     std::cout << "\n===== 附件二车型验证 =====" << std::endl;
752     struct AttachVehicle { std::string name; double L, W, H; };
753     std::vector<AttachVehicle> vehicles = {
754         {"单节柜", 590, 235, 239}, {"集装箱", 1200, 235, 239}, {"超高柜"
            , 1200, 235, 269},
755         {"4.2米箱货", 410, 190, 200}, {"6.2米箱货", 600, 200, 190}, {"6.8米
            箱货", 680, 250, 220},
756         {"9.6米箱货", 950, 250, 220}, {"17米箱货", 1700, 250, 220}
757     };
758     auto cargo_full = generateCargoList();
759     std::vector<const CargoType*> cargo(cargo_full.begin(),
        cargo_full.begin()+100);
760     for (const auto& av : vehicles) {
761         VehicleType v(av.name, av.L, av.W, av.H, 1e6, 0);
762         RealPacker packer(&v);

```



```

763     auto [placed, util] = packer.gaOptimize(cargo, 10);
764     std::cout << av.name << ": 利用率 " << std::fixed << std::
        setprecision(2) << util*100 << "%" << std::endl;
765 }
766 }
767
768 int main() {
769     rng.seed(42);
770
771     auto start = std::chrono::steady_clock::now();
772
773     // 问题一：单车型最少车辆数
774     auto results_p1 = problem1();
775
776     // 问题二：多车型组合优化
777     problem2();
778
779     // 灵敏度分析
780     sensitivity();
781
782     // 附件二验证
783     validateAttachment2();
784
785     // 输出问题一中 V1 第一辆车的利用率
786     if (results_p1.count("V1") && !results_p1["V1"].empty()) {
787         const auto& first_vehicle_placed = results_p1["V1"][0];
788         double vol_used = 0.0;
789         for (const auto& p : first_vehicle_placed) vol_used += p.
            volume();
790         double util = vol_used / VEHICLE_TYPES.at("V1").volume();
791         std::cout << "\n车型1第一辆车利用率: " << std::fixed << std::
            setprecision(1) << util*100 << "%" << std::endl;
792     }
793

```

```

794 // 单车多目标优化测试
795 std::cout << "\n===== 单车多目标优化 (容积+重量最大化) =====" <<
    std::endl;
796 auto all_cargo = generateCargoList();
797
798 for (const auto& vkey : {"V1", "V2"}) {
799     const VehicleType* vehicle = &VEHICLE_TYPES.at(vkey);
800     RealPacker packer(vehicle);
801
802     auto [placed, vol_util, weight_util] = packer.
        optimizeMultiObjective(
803         all_cargo, 50, RealPacker::OptMode::PRODUCT
804     );
805
806     std::cout << "\n车型: " << vehicle->name << std::endl;
807     std::cout << "    目标模式: 容积率 × 重量率 最大化" << std::
        endl;
808     std::cout << "    装载件数: " << placed.size() << " / " <<
        all_cargo.size() << std::endl;
809     std::cout << "    容积利用率: " << std::fixed << std::
        setprecision(2) << vol_util * 100 << "%" << std::endl;
810     std::cout << "    重量利用率: " << weight_util * 100 << "%" <<
        std::endl;
811     std::cout << "    乘积得分: " << std::setprecision(4) <<
        vol_util * weight_util << std::endl;
812
813     std::cout << "    前5件货物坐标示例: " << std::endl;
814     for (size_t i = 0; i < std::min<size_t>(5, placed.size()); ++
        i) {
815         const auto& p = placed[i];
816         std::cout << "        " << p.cargo->id << ": ("
817             << std::fixed << std::setprecision(1)
818             << p.x << ", " << p.y << ", " << p.z << ")" <<
819             << p.l << "x" << p.w << "x" << p.h << std::endl

```

```

;
820     }
821 }
822
823 auto end = std::chrono::steady_clock::now();
824 std::chrono::duration<double> elapsed = end - start;
825 std::cout << "\n全部完成，耗时 " << std::fixed << std::
    setprecision(2) << elapsed.count() << " 秒" << std::endl;
826 return 0;
827 }

```